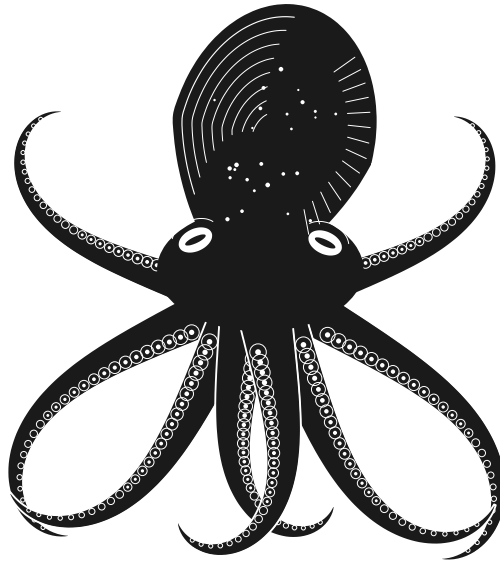


From your first API call to shipping real services



APIs to FastAPI

A Practical Guide

FIRST EDITION

APIs to FastAPI

*A Practical Guide from Your First API Call
to Building and Shipping Real Services*

FIRST EDITION



APIs to FastAPI: A Practical Guide

Copyright © 2026. All rights reserved.

Revision History for the First Edition

2026-09-25: First release

The programs in this book are included for their instructional value. They have been tested with Python 3.14, FastAPI 0.141, Pydantic 2.13, and SQLAlchemy 0.0.47, but no warranty, express or implied, is offered with respect to them, and no liability is accepted for any damages arising from their use. Third-party services and tools described here (GitHub, Postman, Anthropic, and hosting platforms) change over time; check their current documentation.

The complete, tested code for the book's project is in the `bookmarks-api/` folder that accompanies this book.

The cover illustration is an original drawing made for this book.

Typeset in Crimson Pro, Source Sans 3, and Source Code Pro with $\text{\LaTeX}$. Source: Markdown, converted with pandoc.

Contents

Preface.	ix
Who This Book Is For.	ix
How This Book Is Organized	ix
How to Use This Book	ix
What You Need Installed.	x
Conventions Used in This Book	x
 Part I. Understanding APIs	 1
 1. What an API Is	 2
The One-Sentence Version	2
The Restaurant Picture	2
Two Kinds of “API” People Talk About	3
You Already Use APIs All Day.	3
Why APIs Matter (and Why People Pay for Them)	3
Key Vocabulary So Far	4
Exercises	4
Summary, Key Terms, and Review Questions	4
 2. What Actually Happens When You “Make an API Call”	 6
Our Example Call.	6
The URL: The Address of What You Want	6
Finding the Computer: DNS and IP Addresses	7
Ports: Which “Door” on the Computer.	7
localhost: Your Own Computer	8
HTTPS: The Sealed Envelope.	8
The Request Travels	9
The Server Does Its Work.	9
The Response Comes Back	10
The Whole Journey in One Picture	10
How Applications Use This in Practice	10
Where Things Go Wrong (and What the Error Means)	10
Lab: Making and Breaking Your First API Call	12
Summary, Key Terms, and Review Questions	12

3. HTTP and JSON: The Language of API Calls.	14
The Shape of a Request.	14
The Shape of a Response	14
Methods: The Verbs	15
Three Ways to Send Information in a Request	16
Status Codes: Did It Work?	16
Headers: Information About the Message	17
Headers Starting with X-.	18
JSON: The Data Format APIs Speak.	19
Other Body Formats (Just So You Recognise Them)	20
Lab: Reading a Real Response	20
Summary, Key Terms, and Review Questions	21
4. REST, and the Other Styles You'll Hear About.	22
REST: The Style Almost Everyone Uses.	22
REST Conventions Worth Following	23
The Other API Styles, and Whether You Need Them Now	23
Exercises	24
Summary, Key Terms, and Review Questions	24
 Part II. Using APIs	 25
5. Making API Calls Yourself.	26
Watching Real Apps Make API Calls (Browser DevTools)	26
curl: API Calls from the Terminal.	27
Python: API Calls from Code	28
Reading API Documentation	30
Lab: A GitHub Profile Lookup Script	30
Summary, Key Terms, and Review Questions	30
6. Postman.	32
What Postman Is, and Why People Use It	32
Installing Postman	33
The Postman Screen	33
Your First Request	33
Query Parameters: The Params Tab	34
Headers: The Headers Tab	34
Sending a Body: POST, PATCH, and DELETE.	34
Collections: Saving and Organising Requests	35
Variables and Environments	35
Authorization: The Authorization Tab	36
Scripts: Automatic Checks and Chaining Requests.	36
Running a Whole Collection	37
Other Features You'll Use.	37
Postman vs. curl vs. Code vs. FastAPI's Docs Page	37
Lab: A GitHub Explorer Collection	37

Summary, Key Terms, and Review Questions	38
7. Authentication: Proving Who You Are	39
API Keys	39
Basic Authentication	40
Bearer Tokens	40
JWT: What Those Long eyJ... Tokens Are	41
Sessions and Cookies (Briefly)	41
OAuth 2.0: Letting an App Act on Your Behalf	41
Keeping Secrets Out of Your Code	44
Lab: Authenticating to GitHub	44
Summary, Key Terms, and Review Questions	44
8. How Real APIs Behave	46
Pagination: Getting Results a Page at a Time.	46
Rate Limits: Don't Call Too Fast	47
Timeouts and Retries.	47
Idempotency: Retrying POST Safely	48
Reading Error Responses.	49
Webhooks: API Calls That Come to You	49
Lab: Paging Through a Real API	50
Summary, Key Terms, and Review Questions	50
Part III. Building APIs with FastAPI	52
9. The Python You Need First	54
Type Hints	54
Pydantic: Data Validation with Classes	55
Async: Just Enough.	56
Lab: Validating a Signup Form	57
Summary, Key Terms, and Review Questions	57
10. Your First FastAPI App	59
What FastAPI Is (and the Pieces Around It)	59
Setting Up the Project	59
Hello, API.	60
Path Parameters	61
Query Parameters	62
The Free Documentation: /docs	62
Using Your API from Postman	62
Lab: A Temperature Converter Endpoint.	63
Summary, Key Terms, and Review Questions	63
11. Request Bodies, Validation, and Responses	65
Receiving a JSON Body	65
Separate Models for Input and Output.	66

response_model: Controlling What Goes Out	66
Status Codes and Errors	66
Full In-Memory CRUD	67
Trying the Endpoints in Postman	68
A Few More Request Features (for Reference)	69
Lab: Search and Count Endpoints	69
Summary, Key Terms, and Review Questions	70
12. Adding a Database	71
Database Basics in Two Minutes	71
An ORM: SQLAlchemy	71
Growing into Multiple Files	72
database.py: The Connection	72
models.py: The Tables	72
schemas.py: The API Shapes	73
Dependencies: How the Endpoint Gets a Session	74
main.py: CRUD Against the Database	74
Changing Tables Later: Migrations	76
Lab: Adding a Favourites Column	77
Summary, Key Terms, and Review Questions	77
13. Structuring a Real Project	79
Settings from Environment Variables	79
Routers: Splitting Endpoints Across Files	80
Dependencies, Properly	83
The Annotated Style	83
Recording Your Dependencies	84
Lab: A Stats Router	84
Summary, Key Terms, and Review Questions	84
14. Securing Your API	86
Never Store Passwords, Store Hashes	86
The User Table and Schemas	86
security.py: Hashing, Tokens, and “Who Is Calling?”	87
The Auth Router: Register and Log In	89
Protecting Bookmarks and Enforcing Ownership	90
Trying It in /docs	91
Trying It in Postman, with Automatic Token Handling	91
API Keys: The Alternative for Machine Clients	92
CORS: Why Browsers Block Your API (and How to Allow It)	92
A Security Checklist for Every API You Build	93
Lab: A “Who Am I?” Endpoint	93
Summary, Key Terms, and Review Questions	94
15. Calling Other APIs from FastAPI	95
The Plan	95
Background Tasks	95

Calling an AI Model: The Anthropic SDK	96
<code>enrich.py</code> : Fetching Pages and Writing Summaries	96
Async Calls: When They're Worth It	99
Receiving Webhooks	99
Integration Habits (the Checklist Clients Pay For)	101
Lab: A Refresh Endpoint	101
Summary, Key Terms, and Review Questions	101
16. Testing and Shipping	103
Automated Tests with <code>pytest</code> and <code>TestClient</code>	103
Logging: Seeing What Happened	106
Docker: Packaging Your App	106
Deploying	107
Before You Call It Production-Ready	108
Lab: More Tests, Then Deploy	108
Summary, Key Terms, and Review Questions	109
Part IV. From Learner to Earner	110
17. What Clients Pay For, and What to Learn Next	111
The Kinds of API Work People Pay For	111
Your Portfolio: Three Projects	112
How to Approach a Client Project	112
What to Learn Next (in This Order)	113
What to Deliberately Skip for Now.	113
A Six-Week Plan	113
Exercises	114
Summary, Key Terms, and Review Questions	114
Appendices	115
A. Status Codes Cheat Sheet	116
The Status Codes You'll Meet.	116
The Debugging Rule	117
B. Headers Cheat Sheet	118
Request Headers	118
Response Headers	118
The <code>x-</code> Prefix	119
C. FastAPI Cheat Sheet	120
Endpoints, Parameters, and Responses	120
Commands and URLs	121
<code>def</code> vs. <code>async def</code>	121

D. Glossary 122
 Glossary of Terms 122

Index 129

Preface

This book takes you from knowing almost nothing about APIs to building, securing, testing, and shipping your own with FastAPI. This preface explains who it is for, how it is organized, how to get the most out of it, and the conventions it uses.

Who This Book Is For

You know basic Python: variables, functions, lists, dictionaries, loops, `import`, and installing packages with `pip`. You do **not** need to know anything about APIs, networking, web servers, type hints, `async`, or `Pydantic`. Everything else is explained when it's needed, in the order you need it.

How This Book Is Organized

- **Part 1** explains what APIs are and what physically happens when one program talks to another. No coding yet, just a clear mental model. Everything later builds on it.
- **Part 2** makes you a confident API *user*. You'll call real APIs from the terminal, from Postman, and from Python, and learn how logins, tokens, limits, and errors work.
- **Part 3** makes you an API *builder*. You'll build one real project with FastAPI, step by step: a **Bookmarks API** where users sign up, save links, and get an automatic title and AI-written summary for each link. By the end it has a database, logins, tests, and a deployment.
- **Part 4** shows how this skill turns into paid work.

How to Use This Book

Read Parts 1 and 2 in order; they're short. In Part 3, **type the code yourself** rather than copying it. Every chapter ends with **Key takeaways** and a small **Try it** task. Do the tasks: they're where the learning sticks.

A complete, tested copy of the finished project is in the `bookmarks-api/` folder next to this book. Use it to check your work when you're stuck, not to skip the typing.

When you meet a term you don't know, check **Appendix D (Glossary)**. Every term in the book is defined there in a line or two.

What You Need Installed

- **Python 3.12 or newer** (check with `python3 --version`)
- **A code editor**: VS Code is the usual choice
- **Postman**: free, from postman.com (installation is covered in Chapter 6)
- **curl**: already installed on macOS and Linux, and on Windows 10+

You'll install the Python packages as you go.

Conventions Used in This Book

- **Bold** marks a new term the first time it is explained. Every such term is also in Appendix D.
- `Constant width` marks code, commands, file names, URLs, header names, and anything else you would type exactly.
- Code blocks show Python, shell commands, JSON, or program output. Shell commands are shown without a prompt, so you can paste them as they are.
- Three kinds of callout appear in the text:



A suggestion that saves time or trouble.



A general remark, or something to be aware of.



A mistake that can cost you data, money, or security. Read these twice.

- Each chapter ends with a **Lab** (hands-on, with expected results so you can check your work) or **Exercises**, followed by a **Summary, Key Terms, and Review Questions** section.

Understanding APIs



What to ignore for now: networking theory (TCP handshakes, OSI layers), the history of the web, and HTTP/2 vs HTTP/3. None of it will stop you from using or building APIs. You can learn it later if you ever need it.

What an API Is

Before you can use or build an API, you need a clear picture of what one is. This chapter gives you that picture, the vocabulary that goes with it, and a sense of why APIs are worth learning.

The One-Sentence Version

An **API** (Application Programming Interface) is **an agreement about how one program can ask another program to do something or give it something**.

The “interface” part is the important word. An interface is a boundary with rules. A light switch is an interface to your home’s wiring: you don’t need to know how the wiring works, you just flip the switch. An API is the light switch for a piece of software.

The Restaurant Picture

You sit at a table (you are the **client**). The kitchen (the **server**) has the food (the data) and the cooks (the logic). You’re not allowed into the kitchen. Instead:

1. You read the **menu**, which lists what you’re allowed to order. This is the API’s **documentation**.
2. You give your order to the **waiter** in a specific format (“one pasta, no cheese”). This is the **request**.
3. The kitchen prepares it; you never see how.
4. The waiter brings back your food, or tells you “sorry, we’re out of pasta”. This is the **response**.

The waiter is the API. The menu is the contract. You don’t care whether the kitchen uses gas or electric stoves, and the kitchen can change its stoves any time without telling you, as long as the menu stays the same. **That separation is the whole point of APIs**: each side can change its insides freely, as long as the agreement between them holds.

Two Kinds of “API” People Talk About

The word is used loosely, so it helps to separate two meanings:

1. Library APIs (inside your program). When you write `import math` and call `math.sqrt(16)`, you’re using the *math module’s API*: a set of functions with agreed names, inputs, and outputs. Everything happens inside your own program on your own computer.

2. Web APIs (across a network). When a weather app on your phone shows today’s forecast, the app sends a message over the internet to a weather company’s computer, which sends back the forecast data. The two programs are on different machines, often owned by different companies, written in different languages.

This book is about web APIs. When developers say “API” in job listings, freelance projects, or conversations about FastAPI, they almost always mean web APIs. From here on, “API” means web API.

You Already Use APIs All Day

- **Logging in with Google** on some website: that site calls Google’s APIs to confirm who you are.
- **Paying online:** the shop sends your card details to a payment company’s API (such as Stripe’s) and gets back “approved” or “declined”.
- **Maps inside a delivery app:** the app calls a maps API to draw the map and calculate routes.
- **ChatGPT or Claude inside other products:** those products call the AI company’s API, send your text, and get the AI’s reply back.
- **Your phone’s apps in general:** almost every app is a thin screen on top of API calls to its company’s servers. When you scroll Instagram, the app is constantly calling Instagram’s API: “give me the next 10 posts”.

Why APIs Matter (and Why People Pay for Them)

Software today is mostly **built by connecting pieces**. A small startup doesn’t build its own payment system, email system, maps, or AI model. It calls APIs for all of those and writes the glue. That glue is also usually an API: the company’s own backend, which its app and website call.

So there are two skills, and this book teaches both:

- **Using APIs** (being the client): connecting to other people’s services.

- **Building APIs** (being the server): creating the backend that apps, websites, or other businesses call. This is what FastAPI is for.

Key Vocabulary So Far

Term	Meaning
Client	The program making the request (an app, a website, a script, Postman)
Server	The program receiving the request and sending the answer
Request	The message the client sends
Response	The message the server sends back
API call	One request plus its response
Endpoint	One specific “item on the menu”, e.g. “get the weather for a city”
Documentation (docs)	The menu: what you can ask for, and how

Exercises

Pick three apps on your phone. For each, write down two things it probably asks its server for (e.g. Uber: “drivers near me”, “price for this trip”). You’re already thinking in endpoints.

Summary, Key Terms, and Review Questions

Summary

- An API is an agreement between two programs: “send me this, and I’ll send you that.”
- In this book, API means **web API**: programs talking across a network.
- The client asks, the server answers. One request plus one response is one **API call**.
- APIs let each side change its internals freely as long as the agreement holds.

Key Terms

API, Web API, Client, Server, Request, Response, API call, Endpoint, Documentation (API docs). Each term is defined in Appendix D.

Review Questions

1. In your own words, what is an API? Use the restaurant analogy.

2. What is the difference between a library API and a web API?
3. Why can a server change its internals without breaking its clients?
4. What is the difference between an endpoint and an API call?

What Actually Happens When You “Make an API Call”

“Make an API call” sounds abstract. It’s actually a very concrete sequence of steps. This chapter follows one call from start to finish.

Our Example Call

Suppose a program wants the public profile of the GitHub user `octocat`. It makes this call:

```
GET https://api.github.com/users/octocat
```

Let’s take that apart piece by piece, then follow the message across the internet.

The URL: The Address of What You Want

A **URL** (Uniform Resource Locator) is a full address. Here is a longer example with every part labelled:

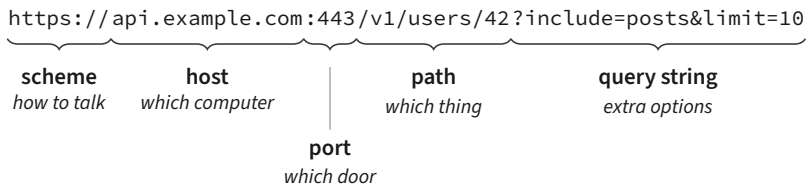


Figure 2-1. The parts of a URL

Part	Example	What it means
Scheme	https	<i>How</i> to talk: HTTPS (encrypted) or HTTP (not encrypted).
Host (domain)	api.example.com	<i>Which computer</i> to talk to.
Port	443	<i>Which door</i> on that computer (explained below). Usually hidden.
Path	/v1/users/42	<i>What thing</i> you want on that computer.
Query string	?include=posts&limit=10	<i>Extra options</i> , as key=value pairs joined by &.

In our GitHub example: the scheme is `https`, the host is `api.github.com`, the path is `/users/octocat`, and there is no query string. The port is hidden, but it's there.

Finding the Computer: DNS and IP Addresses

Computers on the internet don't really find each other by names like `api.github.com`. They use numeric addresses called **IP addresses**, such as `140.82.112.6`. Think of the domain name as a contact name in your phone and the IP address as the actual phone number.

DNS (Domain Name System) is the internet's phone book. Before your program can send anything, it asks a DNS server: "What's the IP address of `api.github.com`?" and gets back a number. This happens automatically, in milliseconds, and is usually cached so it doesn't repeat every time.

You'll rarely deal with DNS directly. But when you see an error like *"Could not resolve host"* or *"Name or service not known"*, it means **this step failed**: the name couldn't be turned into an address, usually because of a typo in the domain or no internet connection.

Ports: Which "Door" on the Computer

One computer can run many programs that accept network connections: a web server, a database, an email server. How does an incoming message know which program it's for? **Ports**.

A port is a number from 0 to 65535. If the IP address is the building's street address, the port is the **apartment number**. Each server program "listens" on a port, meaning it waits for messages arriving at that number.

Ports you'll meet constantly:

Port	Usually used for
443	HTTPS: almost every real API on the internet
80	HTTP (unencrypted)
8000	Your FastAPI app during development (a convention, not a rule)
5432	PostgreSQL database
3000, 5173	Frontend development servers (React, Vite)

When a URL has no port, the default for the scheme is used: 443 for https, 80 for http. So `https://api.github.com/users/octocat` really means `https://api.github.com:443/users/octocat`.

When you run your own API later, you’ll open `http://localhost:8000/docs`. That means: HTTP, to **this computer** (`localhost`), to whatever program is listening on **port 8000**, path `/docs`.

localhost: Your Own Computer

`localhost` is a special name meaning “*this same computer*”. Its IP address is `127.0.0.1`. When you run a FastAPI app on your laptop and visit `http://localhost:8000`, the request never leaves your machine. The client (your browser) and the server (your FastAPI app) are just two programs on the same computer, talking through port 8000.

This is how all API development starts: build and test locally, then later put it on a real server with a real domain.

A common error: “*Connection refused*”. It means you reached the computer, but **nothing is listening on that port**. Usually your server isn’t running, or it’s running on a different port than the one you’re calling.

HTTPS: The Sealed Envelope

The `s` in `https` means the connection is **encrypted** using a protocol called **TLS**. Before any API data is sent, the client and server do a quick setup where:

1. The server proves it really is `api.github.com` by showing a **certificate** (a digital ID card signed by a trusted authority).
2. The two sides agree on encryption keys.

After that, everything (your path, headers, passwords, tokens, data) travels as scrambled bytes. Anyone in between (the café Wi-Fi, your internet provider) sees only that you’re talking to GitHub, not what you’re saying.



Real APIs always use HTTPS. Plain HTTP is fine only for `localhost` during development. If you ever send a password or API key over plain `http://` on the internet, assume it's been stolen.

The Request Travels

Now the client has an IP address, a port, and an encrypted connection. It sends the **HTTP request**, which is plain structured text. (Chapter 3 covers it line by line.) It looks roughly like this:

```
GET /users/octocat HTTP/1.1
Host: api.github.com
Accept: application/json
User-Agent: my-script/1.0
```

This text travels as small packets through a chain of routers across the internet to GitHub's data centre. On the way it often passes through infrastructure on GitHub's side before reaching the actual application:

- A **load balancer** spreads incoming requests across many identical servers so no single one is overwhelmed.
- A **reverse proxy** or **API gateway** may check rate limits, handle the HTTPS encryption, or route `/users/...` to the right internal service.

As an API *user* you don't see any of this. As an API *builder* you'll meet some of it when you deploy (Chapter 16).

The Server Does Its Work

GitHub's application code receives the request and does what any API server does:

1. **Routing:** looks at the method and path (`GET /users/octocat`) and picks the function that handles it.
2. **Checking:** is the request valid? Is the caller allowed? Are they over their rate limit?
3. **Working:** looks up `octocat` in its database.
4. **Responding:** builds a response and sends it back over the same connection.

In Part 3 you will write exactly this kind of code. In FastAPI, step 1 is a decorator like `@app.get("/users/{username}")`, step 2 is mostly automatic, step 3 is your Python function, and step 4 happens when your function returns.

The Response Comes Back

The response is also structured text:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
X-RateLimit-Remaining: 59

{
  "login": "octocat",
  "id": 583231,
  "name": "The Octocat",
  "public_repos": 8
}
```

The first line says it worked (200 OK). Then come **headers** (information *about* the response), then a blank line, then the **body**: the actual data, in a format called **JSON**. Your program reads the status code, then reads the JSON, and does whatever it wanted with the data: shows it on screen, saves it, or makes another call.

The Whole Journey in One Picture

All of that typically takes 50 to 300 milliseconds. **That is what “making an API call” means.**

How Applications Use This in Practice

- A **mobile app** makes API calls to its company’s backend every time you open a screen, pull to refresh, or tap a button.
- A **website** (the JavaScript running in your browser) makes API calls in the background as you use the page. You’ll watch this happen in Chapter 5.
- A **backend server** makes API calls to *other* services: your FastAPI app will call a website to fetch page titles, and call an AI API to write summaries.
- A **script** (Python, cron job, automation) makes API calls to move data between systems. This is a huge amount of real-world freelance work.

It’s always the same pattern: **build a request, send it to a URL, wait, read the response.**

Where Things Go Wrong (and What the Error Means)

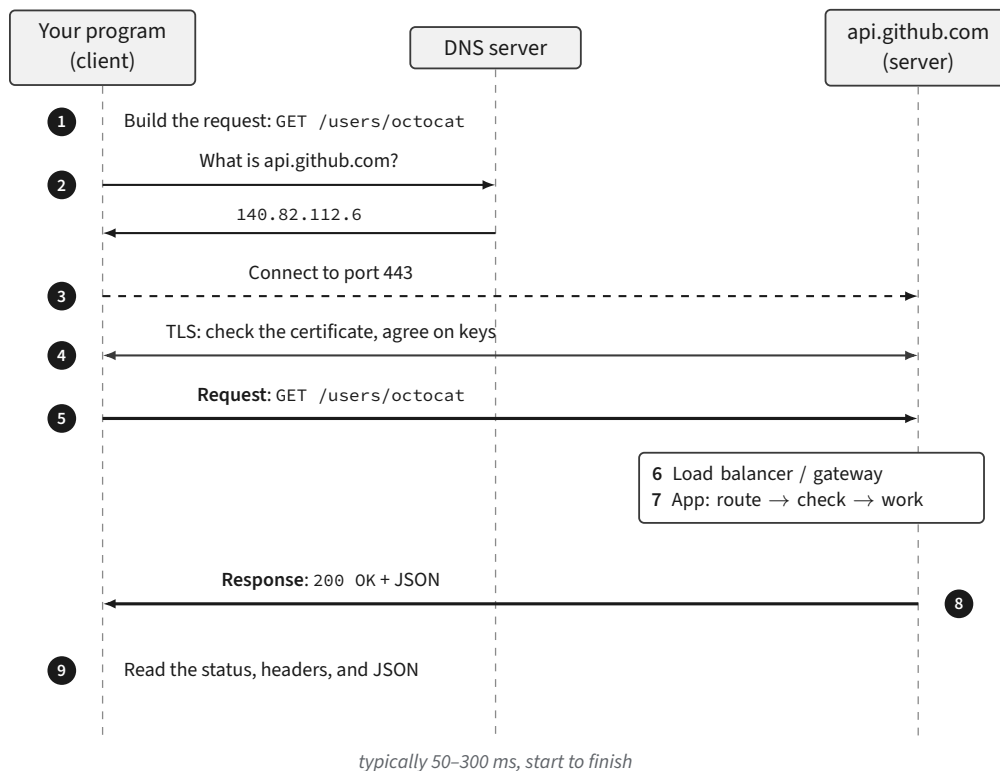


Figure 2-2. The journey of one API call

Symptom	Which step failed	Usual cause
“Could not resolve host”	DNS (step 2)	Typo in the domain, no internet
“Connection refused”	Connect (step 3)	Server not running / wrong port
“Connection timed out”	Connect or response	Server down, firewall, or very slow server
“SSL certificate error”	TLS (step 4)	Expired/self-signed certificate, wrong domain
A 4xx status code	Server checks (step 7)	<i>Your</i> request was wrong (Chapter 3)
A 5xx status code	Server work (step 7)	<i>The server</i> broke

Being able to say “this failed at DNS” instead of “the API doesn’t work” is a large part of what makes someone good at API work.

Lab: Making and Breaking Your First API Call

Open a terminal and run:

```
curl https://api.github.com/users/octocat
```

You just made an API call. Now break it on purpose: try `https://api.githubbbb.com/users/octocat` (DNS failure) and `http://localhost:9999` (connection refused). Read the error messages and match them to the table above.

Expected results: The first command prints JSON containing `"login": "octocat"`. The misspelled domain fails with `Could not resolve host`. The `localhost:9999` call fails with `Failed to connect/Connection refused`, because nothing is listening on that port.

Summary, Key Terms, and Review Questions

Summary

- A URL has a scheme, host, (hidden) port, path, and optional query string.
- DNS turns the host name into an IP address; the port picks which program on that machine.
- `localhost` means “this computer”. Your dev server will live at `http://localhost:8000`.
- HTTPS encrypts everything. Always use it for real APIs.
- An API call is: resolve the name, connect, encrypt, send the request, the server routes, checks, and works, then the response comes back.

Key Terms

URL, Scheme, Host, Port, Path, Query string, DNS, IP address, localhost, HTTPS, TLS, Certificate (TLS), Load balancer, Reverse proxy, API gateway, Routing. Each term is defined in Appendix D.

Review Questions

1. Name the parts of `https://api.example.com:443/v1/users/42?limit=10` and say what each does.
2. What does DNS do, and what error do you see when it fails?
3. What does “Connection refused” tell you, and what is the usual cause during development?
4. Why should you never send an API key over plain `http://` on the internet?
5. What might a request pass through on the server side before it reaches the application?

HTTP and JSON: The Language of API Calls

HTTP (HyperText Transfer Protocol) is the set of rules for what a request and a response look like. Web APIs are built on it. Learn this chapter well and most API documentation becomes readable.

The Shape of a Request

Every HTTP request has up to four parts:

```
POST /v1/bookmarks?notify=true HTTP/1.1    ← 1. Method + path (+ query)
Host: api.example.com                      ← 2. Headers
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
```

← (blank line)

```
{"url": "https://fastapi.tiangolo.com"}    ← 3. Body (optional)
```

1. **Method:** *what kind of action* (here, POST = create).
2. **Path and query:** *what thing* to act on, plus options.
3. **Headers:** *information about the request:* who you are, what format you're sending, what you want back.
4. **Body:** *the data* you're sending. Not every request has one.

The Shape of a Response

```
HTTP/1.1 201 Created                        ← 1. Status code
Content-Type: application/json              ← 2. Headers
Location: /v1/bookmarks/17
```

← (blank line)

```
{"id": 17, "url": "https://fastapi.tiangolo.com"} ← 3. Body
```

1. **Status code:** *did it work*, as a number.
2. **Headers:** *information about the response*.
3. **Body:** *the data coming back*.

That's it. Every API call you'll ever make, in any language, with any tool, is this request/response pair. Tools like curl, Postman, Python's `httpx`, and FastAPI are all just convenient ways to build and read these messages.

Methods: The Verbs

The method tells the server what kind of action you want.

Method	Meaning	Has a body?	Example
GET	Read something	No	GET <code>/bookmarks</code> returns a list of bookmarks
POST	Create something (or trigger an action)	Yes	POST <code>/bookmarks</code> creates a new bookmark
PUT	Replace something entirely	Yes	PUT <code>/bookmarks/17</code> replaces bookmark 17 with the new data
PATCH	Change part of something	Yes	PATCH <code>/bookmarks/17</code> changes just the notes
DELETE	Remove something	Usually no	DELETE <code>/bookmarks/17</code> removes it

Two properties matter in practice:

- **Safe** methods don't change anything on the server. GET is safe. You can call it a thousand times and nothing changes.
- **Idempotent** methods give the same end result whether you call them once or several times. GET, PUT, and DELETE are idempotent (deleting something twice still leaves it deleted). POST is **not**: sending "create a bookmark" twice creates two bookmarks.

Why care? When a request times out, you don't know if the server received it. Retrying a GET or PUT is harmless. Retrying a POST might create a duplicate order or charge a card twice. Chapter 8 returns to this.

PUT vs PATCH confuses everyone at first. PUT sends the *whole* object; anything you leave out may be wiped. PATCH sends *only the fields to change*. Most modern APIs prefer PATCH for updates, and your FastAPI project will use it.

Three Ways to Send Information in a Request

This is one of the most useful distinctions in the whole book. There are three places a client can put information:

1. Path parameters: *which thing*. Part of the path itself, identifying a specific resource.

```
GET /users/octocat      ← "octocat" is a path parameter
GET /bookmarks/17      ← 17 is a path parameter
```

2. Query parameters: *options for how to get it*. After the `?`, as `key=value` pairs, joined by `&`. Used for filtering, searching, sorting, and paging. Always optional-feeling.

```
GET /bookmarks?q=python&limit=10&offset=20
```

3. The body: *the data itself*. Used with `POST`, `PUT`, and `PATCH` to send the content of what you're creating or changing.

```
POST /bookmarks
{"url": "https://example.com", "notes": "read later"}
```

Rule of thumb: **path = which one, query = how, body = what**. (Headers carry a fourth kind of information, *about* the request, covered next.) FastAPI decides which of these your function's arguments come from, as you'll see in Chapters 10 and 11.

Status Codes: Did It Work?

Every response starts with a three-digit number. The **first digit** tells you the category:

Range	Category	Meaning
2xx	Success	It worked
3xx	Redirection	Look somewhere else
4xx	Client error	You (the caller) did something wrong
5xx	Server error	The server broke



The 4xx vs 5xx split is the most important debugging fact in API work. A 4xx means *fix your request*. A 5xx means *it's not your fault; retry later or tell the provider*.

The dozen codes you'll actually see:

Code	Name	When you see it
200	OK	Success, here's your data
201	Created	Your POST created something
204	No Content	Success, nothing to send back (common after DELETE)
301/302	Moved	The resource is at a different URL (clients usually follow automatically)
400	Bad Request	Your request is malformed or makes no sense
401	Unauthorized	You didn't prove who you are (missing/invalid key or token)
403	Forbidden	We know who you are, but you're not allowed to do this
404	Not Found	That thing doesn't exist (or you're not allowed to know it does)
409	Conflict	Clashes with current state (e.g. email already registered)
422	Unprocessable Content	Well-formed but invalid data (FastAPI uses this for validation errors)
429	Too Many Requests	You hit the rate limit; slow down
500	Internal Server Error	The server's code crashed
502/503/504	Bad Gateway / Unavailable / Timeout	Server down, overloaded, or a proxy couldn't reach it

401 vs 403: 401 means “who are you?” (log in first). 403 means “I know who you are, and the answer is no.”

The full list is in Appendix A.

Headers: Information About the Message

Headers are `Name: value` lines that carry **metadata**: information *about* the request or response rather than the data itself. Header names are case-insensitive (`content-type` = `Content-Type`).

You don't need to know hundreds. Here are the ones that matter:

Headers you send (request headers)

Header	Example	Purpose
Content-Type	application/json	“The body I’m sending is JSON.” Required when you send a body.
Accept	application/json	“Please reply in JSON.”
Authorization	Bearer eyJhbG...	Proves who you are (Chapter 7).
User-Agent	my-app/1.0	Which program is calling. Some APIs require it.

Headers you receive (response headers)

Header	Example	Purpose
Content-Type	application/json	The format of the body coming back.
Location	/bookmarks/17	Where the newly created thing lives (with 201).
Retry-After	30	Wait this many seconds before retrying (with 429 or 503).
WWW-Authenticate	Bearer	With 401: how you’re supposed to authenticate.

Headers Starting with X-

You’ll see headers like `X-API-Key`, `X-Request-ID`, or `X-RateLimit-Remaining`. The X- originally meant **“eXtension: a custom, non-standard header”**. Companies invented their own headers for things the official HTTP standard didn’t cover, and marked them with X- to show they were unofficial.

The official advice changed in 2012: new custom headers shouldn’t use X- any more (some later ones just use a company prefix, like `Anthropic-Version`). But thousands of older ones are still in everyday use. So when you see `X-Something`, read it as **“a custom header this particular API (or its infrastructure) uses”**, and look it up in that API’s docs. Common ones:

Header	Meaning
X-API-Key	Your API key, for APIs that use a custom header instead of <code>Authorization</code>
X-Request-ID	A unique ID for this request, for tracing it through logs. Quote it when reporting a bug to a provider.
X-RateLimit-Limit / -Remaining / -Reset	Your rate limit, how many calls you have left, and when it resets
X-Forwarded-For	Added by proxies: the original caller’s IP address

X-Signature, X-Hub-Signature=256

A cryptographic signature proving a webhook is genuine (Chapter 15)

There's nothing magical about them; a header is just a name and a value. The only question is what *this* API expects.

JSON: The Data Format APIs Speak

Almost every modern API sends and receives bodies in **JSON** (JavaScript Object Notation). It's plain text, and if you know Python dictionaries, you already nearly know it.

```
{
  "id": 17,
  "url": "https://fastapi.tiangolo.com",
  "title": "FastAPI",
  "is_favourite": true,
  "rating": 4.5,
  "tags": ["python", "web"],
  "owner": {"id": 1, "email": "asha@example.com"},
  "summary": null
}
```

JSON has only six kinds of values:

JSON	Example	Python equivalent
object	<code>{"a": 1}</code>	<code>dict</code>
array	<code>[1, 2, 3]</code>	<code>list</code>
string	<code>"hello"</code>	<code>str</code>
number	<code>42, 4.5</code>	<code>int, float</code>
boolean	<code>true, false</code>	<code>True, False</code>
null	<code>null</code>	<code>None</code>

Things that trip people up:

- Keys **must** be in double quotes: `{"name": "Asha"}`, not `{name: 'Asha'}`.
- No trailing commas: `[1, 2, 3,]` is invalid JSON.
- `true/false/null` are lowercase (not Python's `True/False/None`).
- **There is no date type.** Dates travel as strings, usually in ISO 8601 format: `"2026-09-25T14:30:00Z"` (the Z means UTC).
- **There are no comments** in JSON.

Converting in Python:

```
import json

text = '{"name": "Asha", "active": true}'
data = json.loads(text)           # JSON text → Python dict
print(data["active"])             # True

back = json.dumps(data)           # Python dict → JSON text
```

In practice you'll rarely call `json` directly. `httpx` gives you `response.json()`, and `FastAPI` converts automatically in both directions.

Other Body Formats (Just So You Recognise Them)

- **Form data** (`application/x-www-form-urlencoded`): `username=asha&password=secret`. What HTML forms send. You'll use it once, for logging in (Chapter 14).
- **Multipart** (`multipart/form-data`): used for **file uploads**.
- **XML**: `<user><name>Asha</name></user>`. Older APIs (banks, government, enterprise). Recognise it; don't study it.

The `Content-Type` header tells the receiver which format the body is in.

Lab: Reading a Real Response

Run this and read the whole output, top to bottom:

```
curl -i https://api.github.com/users/octocat
```

`-i` shows the status line and headers above the body. Find: the status code, the `Content-Type`, and the `x-ratelimit-remaining` header. Run it again. Did the number go down?

Expected results: The first line is `HTTP/2 200` (or `HTTP/1.1 200 OK`), `content-type` is `application/json; charset=utf-8`, and `x-ratelimit-remaining` drops by one on every run (unauthenticated callers get 60 calls an hour).

Summary, Key Terms, and Review Questions

Summary

- A request = method + path/query + headers + optional body. A response = status code + headers + body.
- Methods: GET reads, POST creates, PUT replaces, PATCH partially updates, DELETE removes.
- **Path = which one, query = how, body = what.**
- 2xx = success, 4xx = your fault, 5xx = their fault. Know 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500.
- Headers are metadata. X- headers are custom headers specific to an API.
- JSON is the language of API bodies and maps directly to Python dicts and lists.

Key Terms

HTTP, Method, Safe method, Idempotent, Path parameter, Query parameter, Body, Status code, Header, X- header, Content-Type, JSON, Form data, Multipart. Each term is defined in Appendix D.

Review Questions

1. What are the parts of an HTTP request, and of a response?
2. Why is retrying a POST riskier than retrying a PUT?
3. For “page 2 of Asha’s bookmarks that mention python”, what goes in the path, the query, and the body?
4. What is the difference between 401 and 403? Between 4xx and 5xx?
5. What does an X- prefix on a header tell you?

REST, and the Other Styles You'll Hear About

You now know the mechanics of an API call. This chapter covers the conventions most APIs follow on top of those mechanics, and gives you a short, honest guide to the other API styles you'll hear about, including which ones you can safely ignore for now.

REST: The Style Almost Everyone Uses

REST (Representational State Transfer) is not a technology or a library. It's a **style**, a set of conventions for designing APIs on top of HTTP. When someone says "a REST API" or "RESTful API", they mean an API that follows these conventions (more or less).

The core idea: **your API exposes *resources* (things), and you act on them with HTTP methods.**

A **resource** is any noun your system deals with: users, bookmarks, orders, invoices. Each resource type gets a URL, and the methods from Chapter 3 do the actions:

Action	Method + path	Typical success code
List bookmarks	GET /bookmarks	200
Create a bookmark	POST /bookmarks	201
Get one bookmark	GET /bookmarks/17	200
Update a bookmark	PATCH /bookmarks/17	200
Delete a bookmark	DELETE /bookmarks/17	204

Those five operations (Create, Read, Update, Delete) are called **CRUD**, and a huge share of real-world APIs are mostly CRUD on a handful of resources. That's exactly what you'll build in Part 3.

REST Conventions Worth Following

- **URLs are nouns, not verbs.** The method is the verb.
 - Good: `POST /bookmarks`
 - Avoid: `POST /createBookmark`, `GET /getBookmarks`
- **Use plural nouns for collections:** `/bookmarks`, `/users`.
- **Nest only for clear ownership:** `/users/5/bookmarks` = bookmarks belonging to user 5. Keep nesting shallow (one level).
- **Use query parameters for filtering, sorting, paging:** `/bookmarks?q=python&limit=20`.
- **Use the right status codes:** 201 for created, 404 for missing, and so on.
- **Stateless:** every request carries everything the server needs (including who you are, usually in a header). The server doesn't remember "the last thing you asked". This is what makes APIs easy to scale across many servers.
- **Version your API** when you plan to make breaking changes: `/v1/bookmarks`. Then `v2` can change things without breaking existing clients.

Real APIs bend these rules sometimes. For actions that don't fit CRUD, it's common and fine to use `POST /orders/17/cancel`. Don't be dogmatic; be consistent.

The Other API Styles, and Whether You Need Them Now

You'll hear these terms. Here's enough to hold a conversation and know when each matters.

GraphQL. One endpoint (`POST /graphql`) where the client sends a *query* describing exactly which fields it wants, and gets exactly that back. Invented by Facebook for complex frontends. You'll *use* it with some APIs (GitHub and Shopify offer it). **Learn later, if a project needs it.**

gRPC. A fast, binary protocol from Google, used mostly for services talking to each other *inside* a company. Not human-readable, not browser-friendly. **Skip for now.**

SOAP. An older, XML-based style with strict contracts. Still found in banking, insurance, government, and old enterprise systems. **Only learn it if a client's system forces you to.**

WebSockets. A connection that stays **open** so the server can push messages to the client at any time. Used for chat, live dashboards, multiplayer games. FastAPI supports them. **Learn when you build something real-time.**

Server-Sent Events (SSE) / streaming. The server keeps a response open and sends data in pieces as it's ready. This is how AI chat apps show text appearing word by word. **Worth knowing exists; learn when needed.**

Webhooks. "Don't call us, we'll call you." Instead of you repeatedly asking Stripe "has the

payment gone through yet?”, you give Stripe a URL, and Stripe sends a POST request to *your* API when something happens. Webhooks are just ordinary HTTP requests going the other direction. **You'll need these soon. Covered in Chapters 8 and 15.**

Bottom line: REST over HTTP with JSON is what the overwhelming majority of paid API work uses. Master it first. Everything else is an add-on you can pick up in a weekend when a project calls for it.

Exercises

Design (on paper) the REST endpoints for a simple **to-do list** API: tasks that can be listed, created, viewed, marked done, and deleted, plus a filter for only unfinished tasks. Write each as METHOD /path with the success status code. Compare with the bookmarks table above.

Summary, Key Terms, and Review Questions

Summary

- REST = resources (nouns) in URLs + HTTP methods as verbs + proper status codes.
- CRUD (create, read, update, delete) maps onto POST, GET, PATCH/PUT, DELETE.
- GraphQL, gRPC, SOAP, and WebSockets exist; learn them only when a project needs them.
- Webhooks are important: they're HTTP requests *sent to you* when something happens.

Key Terms

REST, Resource, CRUD, Stateless, Versioning, GraphQL, gRPC, SOAP, WebSocket, SSE (Server-Sent Events), Webhook. Each term is defined in Appendix D.

Review Questions

1. Why should URLs be nouns rather than verbs in a REST API?
2. What does “stateless” mean, and why does it help APIs scale?
3. When are you likely to meet GraphQL or SOAP in real work?
4. How does a webhook differ from polling?

Using APIs



What to ignore for now: HTTPie, Insomnia, and other alternative tools (they do the same job as curl and Postman); Postman's paid team features (mock servers, monitors, flows); SAML and enterprise single sign-on; writing your own OAuth server.

Making API Calls Yourself

Time to make real calls. You'll use three free public APIs that need no sign-up:

API	Base URL	Good for
GitHub	https://api.github.com	Real-world data, real headers, rate limits
JSONPlaceholder	https://jsonplaceholder.typicode.com	A fake API that accepts POST/PATCH/DELETE (it pretends to save)
httpbin	https://httpbin.org	Echoes back exactly what you sent: perfect for seeing your own request

Watching Real Apps Make API Calls (Browser DevTools)

Before making calls yourself, watch a real website make them.

1. Open Chrome (or Edge/Firefox) and go to <https://github.com/octocat>.
2. Open **DevTools**: right-click → **Inspect**, or Cmd+Option+I (Mac) / F12 (Windows).
3. Click the **Network** tab. Click the **Fetch/XHR** filter. This shows only API calls, hiding images and stylesheets.
4. Reload the page, then click around (e.g. the Repositories tab).

Each row is one API call the page made in the background. Click any row:

- **Headers** tab: the URL, method, status code, request headers, and response headers.
- **Payload** tab: what was sent (query parameters or body).
- **Preview / Response** tab: the JSON that came back.

This is a genuinely useful professional skill. When a client says “our dashboard shows the wrong numbers”, opening the Network tab and looking at the actual API responses tells you in 30 seconds whether the problem is the backend (wrong data in the response) or the frontend (right data, displayed wrong). Right-click any request → **Copy** → **Copy as cURL** to replay it in your terminal.

curl: API Calls from the Terminal

curl is a command-line program that sends HTTP requests. It's installed almost everywhere, and API documentation often shows examples in curl. Reading curl commands is a basic skill.

The flags you need:

Flag	Meaning
-i	Include the response status line and headers in the output
-X METHOD	Use this method (default is GET, or POST if you send data)
-H "Name: value"	Add a request header (repeat for several)
-d 'data'	Send this as the request body
-v	Verbose: show everything, including DNS, connection, and TLS steps

A GET request with a query string:

```
curl "https://api.github.com/search/repositories?q=fastapi&sort=stars&per_page=3"
```

Put URLs containing ? or & in quotes, or your shell will misread them.

Seeing the status and headers:

```
curl -i https://api.github.com/users/octocat
```

You may see HTTP/2 200 instead of HTTP/1.1 200 OK. That's a newer version of HTTP; it means exactly the same thing.

Sending a header, and seeing what you sent:

```
curl https://httpbin.org/headers -H "X-API-Key: abc123"
```

httpbin echoes your headers back, so you can see that X-API-Key: abc123 arrived, along with the User-Agent and Accept headers curl added on its own.

A POST with a JSON body:

```
curl -i -X POST https://jsonplaceholder.typicode.com/posts \  
-H "Content-Type: application/json" \  
-d '{"title": "Hello", "body": "My first POST", "userId": 1}'
```

You get back 201 Created and the "created" post with a new id. The \ at the end of a line just continues the command onto the next line.

PATCH and DELETE:

```
curl -X PATCH https://jsonplaceholder.typicode.com/posts/1 \
-H "Content-Type: application/json" \
-d '{"title": "Changed title"}'

curl -i -X DELETE https://jsonplaceholder.typicode.com/posts/1
```

Seeing the whole journey from Chapter 2:

```
curl -v https://api.github.com/users/octocat -o /dev/null
```

Lines starting with `*` show DNS resolution, the connection to port 443, and the TLS certificate check. Lines with `>` are your request, and lines with `<` are the response headers. Chapter 2's diagram, live.

Python: API Calls from Code

Most real work happens in code. Python's modern HTTP library is **httpx**. (You'll also see the older `requests` library everywhere; it looks almost identical.)

```
pip install httpx
```

GET and read JSON:

```
import httpx

response = httpx.get("https://api.github.com/users/octocat", timeout=10)

print(response.status_code)           # 200
print(response.headers["content-type"]) # application/json; charset=utf-8
data = response.json()                 # JSON body → Python dict
print(data["name"], data["public_repos"]) # The Octocat 8
```

Query parameters: pass a dict and `httpx` builds the `?q=...&sort=...` part for you (and escapes special characters correctly):

```
response = httpx.get(
    "https://api.github.com/search/repositories",
    params={"q": "fastapi", "sort": "stars", "per_page": 3},
    timeout=10,
)
for repo in response.json()["items"]:
    print(repo["full_name"], repo["stargazers_count"])
```

POST a JSON body: `json=` converts your dict to JSON *and* sets Content-Type: application/json:

```
new_post = {"title": "Hello", "body": "My first POST", "userId": 1}
response = httpx.post("https://jsonplaceholder.typicode.com/posts", json=new_post,
    ↪ timeout=10)
print(response.status_code)      # 201
print(response.json())           # {'title': 'Hello', ..., 'id': 101}
```

Headers:

```
response = httpx.get(
    "https://httpbin.org/headers",
    headers={"X-API-Key": "abc123", "User-Agent": "learning-apis/1.0"},
    timeout=10,
)
print(response.json())
```

Handling errors properly. A 404 or 500 is not a Python exception by default. `httpx` happily returns the response. You must check:

```
response = httpx.get("https://api.github.com/users/this-user-does-not-exist-xyz",
    ↪ timeout=10)

if response.status_code == 404:
    print("No such user")
else:
    response.raise_for_status()  # raises httpx.HTTPStatusError for any 4xx/5xx
    print(response.json()["name"])
```

And network failures (DNS, refused, timeout) *do* raise exceptions:

```
try:
    response = httpx.get("https://api.github.com/users/octocat", timeout=10)
    response.raise_for_status()
except httpx.TimeoutException:
    print("The server took too long")
except httpx.HTTPStatusError as error:
    print("Server said no:", error.response.status_code)
except httpx.RequestError as error:
    print("Couldn't reach the server:", error)
```



Always set a timeout. Without one, a hung server can freeze your program. (`httpx` defaults to 5 seconds, but be explicit.)

Reading API Documentation

Every API's docs answer the same questions. When you open unfamiliar docs, look for these, in this order:

1. **Base URL:** e.g. `https://api.github.com`. Every endpoint path is appended to this.
2. **Authentication:** how to prove who you are (key? token? OAuth?) and which header to put it in.
3. **Endpoints:** the list of METHOD /path operations, each with its parameters (path, query, body) and example responses.
4. **Errors:** what error responses look like.
5. **Rate limits:** how many calls you're allowed.
6. **Pagination:** how to get page 2.
7. **SDKs:** many big APIs publish an official Python library (an **SDK**, Software Development Kit) that wraps the HTTP calls for you. Stripe, Anthropic, OpenAI, and Twilio all do. Use the SDK when one exists; it handles auth headers, retries, and errors. But know that underneath, it's making exactly the HTTP calls you just learned.

Many docs are generated from an **OpenAPI** file (a machine-readable description of every endpoint). FastAPI generates one for your API automatically, as you'll see in Chapter 10.

Lab: A GitHub Profile Lookup Script

Write a Python script that asks for a GitHub username, calls `GET /users/{username}`, and prints their name, number of public repos, and account creation date. Handle the “user not found” case with a friendly message. Then add a second call to `GET /users/{username}/repos` and print the names of their repos.

Expected results: For octocat the script prints The Octocat, 8 public repos, and 2011-01-25T18:44:36Z. For a made-up username it prints your friendly message instead of crashing.

Summary, Key Terms, and Review Questions

Summary

- The browser's Network tab shows every API call a website makes. Use it to debug.
- `curl: -i` shows headers, `-X` sets the method, `-H` adds headers, `-d` sends a body, `-v` shows everything.
- In Python, use `httpx: params=` for query strings, `json=` for bodies, `headers=` for headers, and always a `timeout=`.

- A 4xx/5xx is not an exception unless you call `raise_for_status()`. Network failures are.
- When reading API docs: base URL, auth, endpoints, errors, limits, pagination, SDK.

Key Terms

DevTools, **curl**, **httpx**, **Timeout**, **Base URL**, **SDK**, **OpenAPI**. Each term is defined in Appendix D.

Review Questions

1. How can you see the API calls a website makes?
2. Which curl flags show response headers, set the method, add a header, and send a body?
3. In httpx, does a 404 response raise an exception? What does?
4. Which questions should you answer first when reading unfamiliar API documentation?

Postman

Postman is the tool you'll spend the most time in when exploring and testing APIs. This chapter explains what it is and why people use it, then walks through every feature you need: requests, collections, variables, authorization, and scripts.

What Postman Is, and Why People Use It

Postman is a desktop app for making API calls through a visual interface instead of code or the terminal. It does exactly what `curl` and `httpx` do: builds HTTP requests and shows responses. But it adds things that matter when you work with APIs daily:

- **A form instead of a command line:** fill in the method, URL, headers, and body in tabs, with no quoting mistakes.
- **Nicely formatted responses:** pretty JSON, status, time taken, size.
- **Saved requests:** organise requests into **collections** and reuse them tomorrow.
- **Variables and environments:** switch the same requests between `localhost` and production with one click.
- **Built-in auth helpers:** API keys, bearer tokens, and even full OAuth 2.0 flows.
- **Automated checks:** small scripts that verify responses, run in sequence.
- **Sharing:** send a whole collection to a teammate or client.

In practice: developers use Postman to **explore** an API before writing code against it, to **test** their own API while building it, to **debug** ("is it my code or the API?"), and to **hand over** a working set of example requests to clients. Freelance clients often ask for "a Postman collection" alongside an API. Now you'll know what that means.

Alternatives that do the same job: Insomnia, Bruno (open source, stores collections as files), and Thunder Client (inside VS Code). The concepts transfer exactly.

Installing Postman

Download from postman.com/downloads and install the desktop app. Postman asks you to create a free account; that's needed for saving collections and syncing. (Without signing in, a limited "lightweight" mode lets you send one-off requests.) You'll land in a **workspace**: a space holding your collections and environments.

The Postman Screen

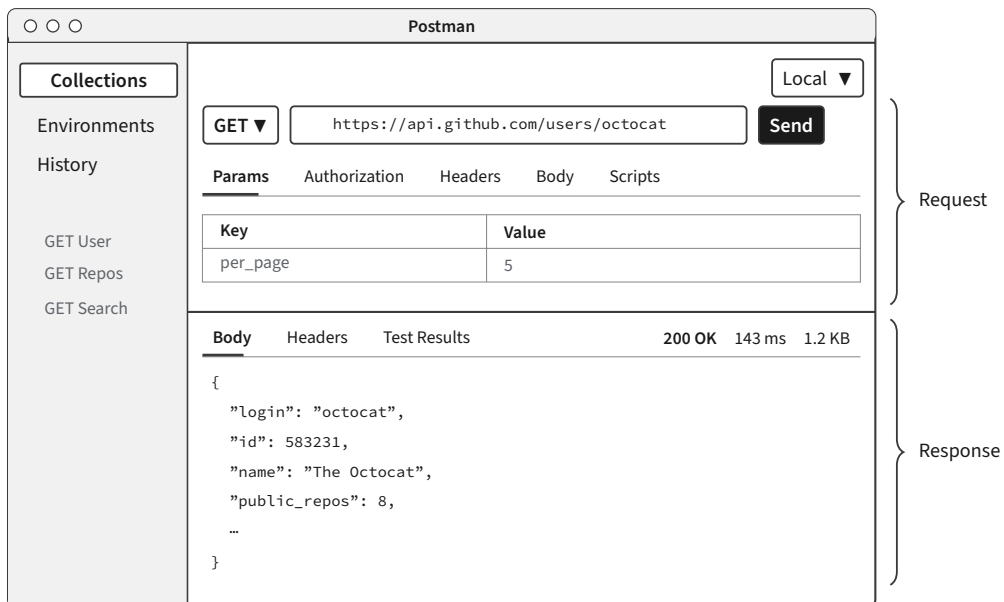


Figure 6-1. The Postman window: the request on top, the response below

The top half is the **request** (everything from Chapter 3: method, URL, params, headers, body). The bottom half is the **response** (status, headers, body). That's the whole model.

Your First Request

1. Click **New** → **HTTP** (or the + tab).
2. Leave the method as **GET**. Enter `https://api.github.com/users/octocat`.
3. Click **Send**.

At the bottom: status 200 OK, the time taken, and the JSON body, formatted. Click the response **Headers** tab and find `x-ratelimit-remaining`.

Query Parameters: The Params Tab

Enter the URL `https://api.github.com/search/repositories`, open the **Params** tab, and add rows:

Key	Value
q	fastapi
sort	stars
per_page	3

Watch the URL bar update itself to `...?q=fastapi&sort=stars&per_page=3`. The Params table and the URL are two views of the same thing. Untick a row to temporarily remove that parameter.

Headers: The Headers Tab

Change the URL to `https://httpbin.org/headers`. In **Headers**, add `X-API-Key = abc123`. Send. `httpbin` echoes it back. Click **hidden** headers to see the ones Postman adds automatically (`User-Agent: PostmanRuntime/...`, `Accept`, and others).

Sending a Body: POST, PATCH, and DELETE

1. Set the method to **POST** and the URL to `https://jsonplaceholder.typicode.com/posts`.
2. Open **Body** → select **raw** → choose **JSON** in the dropdown on the right. (This also sets the `Content-Type: application/json` header for you.)
3. Enter:

```
{ "title": "Hello from Postman", "body": "Testing", "userId": 1 }
```

4. Send. Expect 201 Created.

The other Body options map to Chapter 3's formats: **form-data** (multipart, for file uploads), **x-www-form-urlencoded** (form data, used for login forms), **raw** (JSON, XML, text), **binary** (a file as the whole body).

Now try PATCH on `/posts/1` with `{ "title": "Changed" }` and DELETE on `/posts/1`.

Collections: Saving and Organising Requests

A **collection** is a folder of saved requests. Create one:

1. Sidebar → **Collections** → + → name it `JSONPlaceholder Practice`.
2. On any open request, click **Save** (Cmd/Ctrl+S), give it a clear name like `Create post`, and choose the collection.

Organise with folders inside collections (e.g. `Posts`, `Users`). A good collection reads like the API's menu: every endpoint, each with a working example.

Collections can also hold settings shared by every request inside them, like authorization (next sections) and variables.

Variables and Environments

Hard-coding `https://jsonplaceholder.typicode.com` into every request is painful, especially when the same API has a local version (`http://localhost:8000`) and a live one (`https://api.myapp.com`).

Variables fix this. Write `{{variable_name}}` anywhere (URL, headers, body) and Postman substitutes the value.

An **environment** is a named set of variable values. Create two:

1. Sidebar → **Environments** → + → name it `Local`. Add variable `base_url = http://localhost:8000`.
2. Create another called `Production` with `base_url = https://api.myapp.com`.

Now write your request URLs as `{{base_url}}/bookmarks`, and switch between environments with the selector in the top-right. Same requests, different server, one click. You'll use exactly this setup in Part 3.

Postman has several variable **scopes**: **global** (everywhere), **collection** (inside one collection), and **environment** (whichever environment is selected). Environment variables are the ones you'll use most.

Secrets in variables. Postman can sync your variables to its cloud (for sharing with a team). Each variable has a synced/shared value and a local-only value; the exact labels differ between Postman versions. Put real API keys and passwords **only** in the local value, or in Postman's **Vault** (a local store for secrets). Mark sensitive variables as **secret** type so they're masked on screen.

Authorization: The Authorization Tab

Instead of typing auth headers by hand, use the **Authorization** tab and pick a type:

- **API Key:** enter the key name (e.g. X-API-Key), the value, and whether to add it to the header or the query string.
- **Bearer Token:** paste a token; Postman sends `Authorization: Bearer <token>`.
- **Basic Auth:** username + password (Postman encodes them for you).
- **OAuth 2.0:** fill in the client ID, client secret, and URLs, and Postman runs the whole flow for you and gets a token (Chapter 7 explains what that means).

Set authorization **once on the collection** and choose **Inherit auth from parent** on each request, so every request uses it automatically. Use a variable for the value: `{{token}}`.

Scripts: Automatic Checks and Chaining Requests

The **Scripts** tab lets you run small bits of JavaScript before a request is sent (**Pre-request**) or after the response arrives (**Post-response**). You need only two patterns.

Pattern 1: check the response (a test). In **Post-response**:

```
pm.test("Status is 201", function () {
  pm.response.to.have.status(201);
});

pm.test("Response has an id", function () {
  const body = pm.response.json();
  pm.expect(body.id).to.be.a("number");
});
```

After sending, the response's **Test Results** tab shows pass/fail for each check.

Pattern 2: save a value for the next request. This is how you chain a login into later calls. On a login request's **Post-response**:

```
const body = pm.response.json();
pm.environment.set("token", body.access_token);
```

Now every request using `{{token}}` automatically has a fresh token after you log in. You'll do exactly this with your own API in Chapter 14.

Running a Whole Collection

Click the collection's `...` → **Run** to open the **Collection Runner**. It sends every request in order and reports all test results. That's a quick smoke test of an entire API: "Did everything still work after my change?" (There's also a command-line runner, **Newman** / Postman CLI, for running collections in automated pipelines. You don't need it yet.)

Other Features You'll Use

- **Code snippets:** the `</>` icon on the right turns the current request into curl, Python (requests), JavaScript, and more. Great for moving from "works in Postman" to "works in my code".
- **Import:** paste a curl command (e.g. from DevTools' "Copy as cURL") or an **OpenAPI** URL. Postman builds the requests for you. In Chapter 10 you'll import your own FastAPI app with one URL.
- **History:** every request you've sent, in the sidebar.
- **Console** (bottom bar): the raw request and response exactly as sent. Useful when something mysterious happens.

Postman vs. curl vs. Code vs. FastAPI's Docs Page

Use	When
Postman	Exploring an API, building up a saved, shareable set of requests, testing flows with auth
curl	Quick one-offs, sharing a reproducible command in a bug report or chat, servers without a GUI
Python (httpx)	Anything that's part of your actual program or automation
FastAPI /docs	Quickly trying <i>your own</i> FastAPI endpoints in the browser while developing

Lab: A GitHub Explorer Collection

Build a collection called `GitHub Explorer` with an environment containing `base_url = https://api.github.com` and `username = octocat`. Add three saved requests: get the user, list their repos (with a `per_page` param), and search repositories. Add a test to each that checks for status 200. Run the whole collection with the Runner.

Expected results: The Runner's summary shows three passing tests. If a request fails with a URL error, check that your environment is selected in the top-right corner.

Summary, Key Terms, and Review Questions

Summary

- Postman builds HTTP requests (method, URL, params, headers, body) and shows responses, visually.
- **Collections** save and organise requests; **environments** plus `{{variables}}` switch between servers.
- The **Authorization** tab handles API keys, bearer tokens, Basic auth, and OAuth 2.0.
- **Post-response scripts** check responses (`pm.test`) and pass values between requests (`pm.environment.set`).
- Keep secrets in local-only variable values or the Vault, never in shared ones.

Key Terms

Postman, **Workspace (Postman)**, **Collection (Postman)**, **Environment (Postman)**, **Variable (Postman)**, **Script (Postman)**, **Collection Runner (Postman)**. Each term is defined in Appendix D.

Review Questions

1. What problems do collections and environments solve?
2. Where should secrets live in Postman, and why?
3. How do you pass a token from a login request to later requests automatically?
4. When would you choose curl over Postman?

Authentication: Proving Who You Are

Most useful APIs need to know **who is calling**, so they can decide what you're allowed to do, count your usage, and bill you. Two words that sound alike:

- **Authentication:** *who are you?* (Proving identity. Failure → **401**.)
- **Authorization:** *what are you allowed to do?* (Checking permissions. Failure → **403**.)

There are several ways to authenticate, from simple to sophisticated. You'll see all of them.

API Keys

An **API key** is a long random string the provider gives you, like `sk_live_4eC39HqLyjWDarjtT1zdp7dc`. You send it with every request. The server looks it up and knows who you are.

Where it goes depends on the API; the docs tell you:

<code>Authorization: Bearer sk_live_4eC39Hq...</code>	← common (Stripe, OpenAI)
<code>X-API-Key: 4eC39Hq...</code>	← common custom header
<code>x-api-key: sk-ant-...</code>	← Anthropic's API
<code>GET /weather?city=Paris&appid=4eC39Hq...</code>	← in the query string (older/simpler APIs)

Keys in the query string are the weakest option, because URLs get saved in server logs and browser history. Prefer headers when you have the choice.



API keys are passwords. Anyone with your key *is you* as far as the API is concerned. They can run up your bill or read your data.

A real example: calling Anthropic's Claude API with curl. Notice the custom headers `x-api-key` and `anthropic-version`:

```
curl https://api.anthropic.com/v1/messages \
-H "x-api-key: $ANTHROPIC_API_KEY" \
-H "anthropic-version: 2023-06-01" \
-H "content-type: application/json" \
-d '{
  "model": "claude-opus-5",
  "max_tokens": 1024,
  "messages": [{"role": "user", "content": "Explain an API in one sentence."}]
}'
```

`$ANTHROPIC_API_KEY` reads the key from an environment variable in your terminal (set with `export ANTHROPIC_API_KEY=...`), so the key never appears in the command itself. That's a good habit.

Basic Authentication

The oldest method: send a username and password in a header, as `username:password` encoded with **Base64**:

```
Authorization: Basic YXNoYTpzZWNYZXQxMjM=
```

Base64 is not encryption. It's just a way to write any text safely using only letters and digits; anyone can decode it instantly. Basic auth is only safe over HTTPS. Some APIs use it with an API key as the “username” (and an empty password). You'll mostly meet it with older systems.

Bearer Tokens

A **token** is a string that proves you've *already* authenticated. The flow is:

1. You log in once (with a password, or through OAuth).
2. The server gives you a **token**, usually valid for a limited time (minutes to hours).
3. You send the token with every following request:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

“Bearer” means **whoever holds (bears) this token is let in**. Like a concert ticket, it's not tied to your face. Protect it like a password. Tokens are better than sending passwords every time, because they **expire**: a stolen token is only useful for a short while.

JWT: What Those Long eyJ... Tokens Are

Many bearer tokens are **JWTs** (JSON Web Tokens, pronounced “jot”). A JWT has three parts separated by dots:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MTIiImV4cCI6MTc5MDAwMDAwMH0.K7pX...
```

└── header ─┘ └────────── payload ─────────┘ └── signature ┘

- **Header:** which signing algorithm was used.
- **Payload:** the **claims**, JSON data such as `{"sub": "42", "exp": 17900000000}`. `sub` (subject) is who the token is for (user 42); `exp` is when it expires (a Unix timestamp: seconds since 1 January 1970).
- **Signature:** a cryptographic seal made with a secret key only the server knows.

Two crucial facts:

1. **The payload is readable by anyone.** It’s only Base64-encoded. Paste any JWT into jwt.io and you’ll see its contents. **Never put secrets in a JWT.**
2. **The payload can’t be changed without breaking the signature.** If someone edits `"sub": "42"` to `"sub": "1"`, the signature no longer matches and the server rejects it. That’s what makes JWTs trustworthy.

The advantage: the server can verify a JWT **without a database lookup**. It just checks the signature and expiry. You’ll issue and verify JWTs yourself in Chapter 14.

Sessions and Cookies (Briefly)

Traditional websites log you in differently: the server stores a **session** record and gives your browser a **cookie** (a small value the browser automatically sends back with every request to that site). It achieves the same thing as a bearer token, but the browser handles it automatically. You’ll meet this if you build APIs for a website on the same domain. For APIs used by mobile apps, scripts, and other servers, bearer tokens are standard.

OAuth 2.0: Letting an App Act on Your Behalf

This is where terms like **client ID** and **client secret** come from.

The problem OAuth solves. You use a scheduling app and want it to read your Google Calendar. The terrible solution: give the app your Google password. Then it can read your email, delete your files, and change your password, forever.

OAuth’s solution: the app sends you to **Google**, you log in *on Google’s own page* and approve “this app may read your calendar”. Google then gives the app a **limited, expiring token**, not

your password. You can revoke it any time from your Google account settings.

The cast:

Role	In our example
Resource owner	You (it's your calendar)
Client	The scheduling app that wants access
Authorization server	Google's login and consent system
Resource server	Google Calendar's API, which holds the data

The credentials:

- **Client ID:** the app's public identifier, like a username for the *app* (not for you). E.g. `8123-abc.apps.googleusercontent.com`. It's fine for this to be seen.
- **Client secret:** the app's password. It proves the request really comes from that app. **It must stay on a server, never in a mobile app, frontend JavaScript, or a public repo.**
- **Redirect URI:** the URL on the app where Google sends you back after you approve. It's registered in advance so attackers can't redirect tokens to themselves.
- **Scopes:** the specific permissions requested, like `calendar.readonly`. This is what you see on the consent screen ("This app wants to: see your calendar").
- **Access token:** the short-lived token (often ~1 hour) used to call the API, as a bearer token.
- **Refresh token:** a long-lived token used to get new access tokens without asking you to log in again. Store it securely.

The developer gets the client ID and secret by registering their app in the provider's developer console (Google Cloud Console, GitHub Developer Settings, and so on).

The Authorization Code flow (the standard one, used whenever a user is involved):

The one-time **code** in step 3 passes through your browser, where it could be seen, so it's useless on its own. It only becomes a token in step 4, when combined with the client secret that never leaves the app's server. Apps that can't keep a secret (mobile apps, single-page web apps) use an extra protection called **PKCE** (pronounced "pixie"); libraries handle it for you.

The Client Credentials flow (no user involved, machine-to-machine): your server sends its client ID and client secret directly to the provider's token URL and gets an access token back. Used for server-to-server integrations like "our backend syncs orders with the shipping company's API".

```
response = httpx.post(
    "https://auth.example.com/oauth/token",
    data={
```

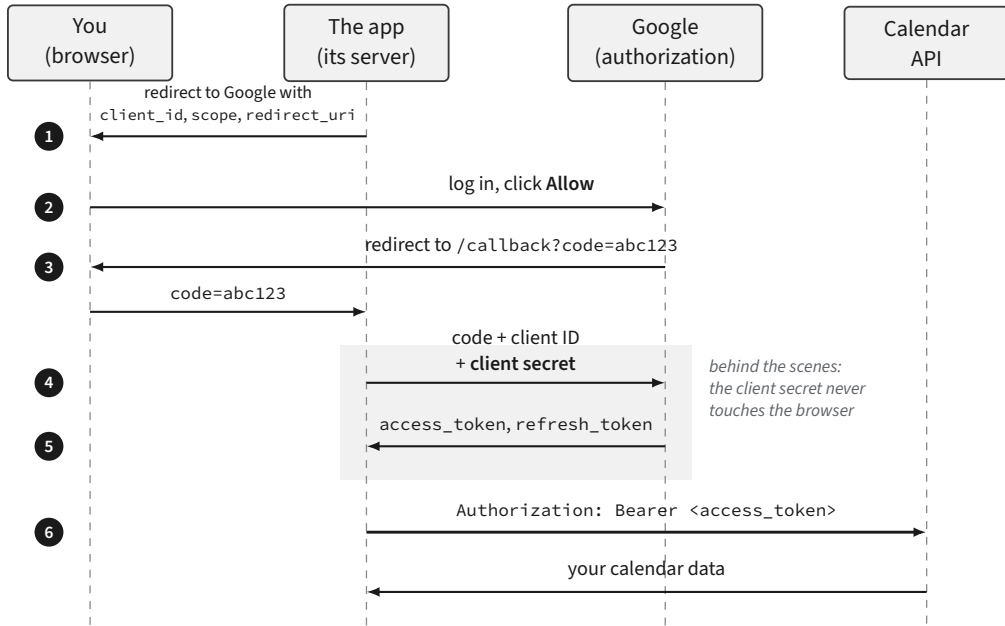


Figure 7-1. The OAuth 2.0 Authorization Code flow

```

    "grant_type": "client_credentials",
    "client_id": CLIENT_ID,
    "client_secret": CLIENT_SECRET,
    "scope": "orders:read",
  },
  timeout=10,
)
access_token = response.json()["access_token"]

```

“Log in with Google” is OAuth plus a thin extra layer called **OpenID Connect (OIDC)**, which additionally gives the app an **ID token** (a JWT) saying who you are. When you want to add social login to a product, you’ll use a library or a service (Auth0, Clerk, Supabase Auth, Firebase Auth) rather than implementing it by hand.

Practical reality: as an API user, you’ll mostly copy a client ID and secret from a developer console, then let Postman’s OAuth 2.0 helper or the provider’s SDK run the flow. You need to *understand* the flow to set it up and debug it, not to reimplement it.

Keeping Secrets Out of Your Code

The most common real-world security failure isn't clever hacking. It's **keys committed to GitHub**. Bots scan public repositories constantly and use leaked keys within minutes.

The standard practice:

1. Put secrets in a file named `.env` in your project folder:


```
GITHUB_TOKEN=ghp_XXXXXXXXXXXXX
ANTHROPIC_API_KEY=sk-ant-XXXXXXXXX
```
2. Add `.env` to your `.gitignore` file so Git never commits it.
3. Commit a `.env.example` with the variable *names* but fake values, so others know what to set.
4. Read the values in your code (FastAPI projects use `pydantic-settings`, in Chapter 13; plain scripts can use `python-dotenv` or `os.environ`).
5. In production, set the same variables in your hosting platform's settings screen.



If a key ever leaks, revoke it in the provider's dashboard immediately and create a new one. Deleting the commit is not enough; assume it's already copied.

Lab: Authenticating to GitHub

1. Create a GitHub **personal access token** (GitHub → Settings → Developer settings → Personal access tokens → fine-grained, read-only access is fine).
2. Call GET `https://api.github.com/user` in Postman with no auth. You should get 401.
3. Add the token under Authorization → Bearer Token. Send again: 200, with *your* profile.
4. Compare the `x-ratelimit-limit` header with and without the token. Authenticated callers get a much higher limit.
5. Paste any JWT you find (e.g. from `jwt.io`'s example) into `jwt.io` and read its payload.

Expected results: Step 2 returns 401 with "message": "Requires authentication". Step 3 returns 200 with your own profile. In step 4, `x-ratelimit-limit` rises from 60 to 5000.

Summary, Key Terms, and Review Questions

Summary

- **Authentication** = who you are (401 if missing). **Authorization** = what you may do (403 if denied).
- API keys are passwords. Send them in headers, never commit them.

- Bearer tokens: log in once, get an expiring token, send `Authorization: Bearer <token>`.
- JWTs are readable by anyone but can't be tampered with. Never put secrets inside one.
- OAuth 2.0 lets an app act for a user without their password. Client ID = the app's public name, client secret = the app's password (server-only), scopes = permissions, access token = short-lived key, refresh token = gets new access tokens.
- Secrets live in `.env` (git-ignored) locally and in environment variables in production.

Key Terms

Authentication, Authorization, API key, Basic auth, Base64, Bearer token, JWT, Claim, Session (web), Cookie, OAuth 2.0, Client ID, Client secret, Redirect URI, Scope, Access token, Refresh token, Authorization Code flow, Client Credentials flow, PKCE, OpenID Connect (OIDC), Environment variable. Each term is defined in Appendix D.

Review Questions

1. Define authentication and authorization, and give the status code for each failure.
2. Why is Base64 not encryption, and what does that mean for Basic auth?
3. Anyone can read a JWT's payload. Why can't they change it?
4. In OAuth, what is the client secret for, and where must it never appear?
5. Walk through the Authorization Code flow in your own words.

How Real APIs Behave

Tutorials show the happy path. Real APIs return thousands of records, limit your calls, fail sometimes, and send you events. This chapter covers what you'll meet on every real integration.

Pagination: Getting Results a Page at a Time

No API returns a million records in one response. It returns a **page** and tells you how to get the next. Three common styles:

1. Offset/limit (or page/per_page): “skip N, give me M”.

```
GET /bookmarks?offset=0&limit=20      ← first 20
GET /bookmarks?offset=20&limit=20     ← next 20
GET /repos?page=2&per_page=50        ← GitHub's version
```

Simple, and lets you jump to any page. It can skip or repeat items if records are added while you're paging. Your FastAPI project uses this style.

2. Cursor-based: the response includes a pointer to “where you left off”.

```
{
  "data": [ ... ],
  "next_cursor": "eyJpZCI6MTAwfQ",
  "has_more": true
}
```

You send `?cursor=eyJpZCI6MTAwfQ` to get the next page, and stop when `has_more` is false. More reliable for large or fast-changing data; Stripe, Slack, and many modern APIs use it.

3. Link header: the next page's URL is in a response header, e.g. GitHub's:

```
Link: <https://api.github.com/user/repos?page=2>; rel="next", <...?page=5>;
     ↪ rel="last"
```

A typical “get everything” loop:

```
import httpx

def fetch_all_repos(username: str) -> list[dict]:
    repos, page = [], 1
    while True:
        response = httpx.get(
            f"https://api.github.com/users/{username}/repos",
            params={"per_page": 100, "page": page},
            timeout=10,
        )
        response.raise_for_status()
        batch = response.json()
        if not batch:
            return repos
        repos.extend(batch)
        page += 1
```

Rate Limits: Don't Call Too Fast

APIs cap how many calls you can make per minute, hour, or day, to protect their servers and enforce pricing tiers. Exceed it and you get **429 Too Many Requests**.

Signs to watch for:

- Response headers like `X-RateLimit-Limit: 5000`, `X-RateLimit-Remaining: 12`, `X-RateLimit-Reset: 1790361130` (a Unix timestamp for when your quota refills).
- On a 429 (or 503), a `Retry-After` header telling you how many seconds to wait.

How to behave well:

- Check `Remaining` and slow down before hitting zero.
- On 429, wait for `Retry-After` if given, then retry.
- Don't make calls you don't need. Cache results that don't change often.

Timeouts and Retries

Networks fail and servers hiccup. A reliable API client:

1. **Always sets a timeout.** Never wait forever.
2. **Retries only what's worth retrying:**
 - Network errors, timeouts, **429**, and **5xx**: temporary, so retry.

- **4xx** (other than 429): your request is wrong; retrying the same thing will fail the same way, so fix it instead.
3. **Waits longer between each retry (exponential backoff)**: 1s, 2s, 4s, 8s... plus a little randomness (**jitter**) so thousands of clients don't all retry at the same instant.
 4. **Gives up** after a few attempts and reports the error.

```
import random
import time

import httpx

def get_with_retries(url: str, attempts: int = 4) -> httpx.Response:
    for attempt in range(attempts):
        try:
            response = httpx.get(url, timeout=10)
            if response.status_code != 429 and response.status_code < 500:
                return response          # success, or an error retrying won't
                ↪ fix
        except httpx.TransportError:
            pass                        # network problem: worth retrying
            time.sleep(2 ** attempt + random.random()) # 1s, 2s, 4s... plus jitter
    raise RuntimeError(f"Gave up on {url} after {attempts} attempts")
```

(Libraries like `tenacity` do this for you, and good SDKs such as Stripe's or Anthropic's retry automatically.)

Idempotency: Retrying POST Safely

Remember from Chapter 3: retrying a POST might create duplicates. If your “charge the customer” request timed out, did the charge happen or not? You can't know.

Payment APIs solve this with an **idempotency key**: a unique ID you generate and send with the request:

```
POST /v1/charges
Idempotency-Key: 5f1c2a9e-8d4b-4c3a-9f7e-2b6d1e0a3c55
```

If the server sees the same key twice, it doesn't charge again; it returns the original result. Generate one per logical operation (Python: `str(uuid.uuid4())`) and reuse it on retries. Any time money or irreversible actions are involved, look for this in the API docs.

Reading Error Responses

When a call fails, **read the body**. Most APIs explain what went wrong in JSON. Formats vary:

```
{"detail": "Bookmark not found"}
```

```
{"error": {"type": "invalid_request_error", "message": "Missing required param:  
↪ amount"}}}
```

```
{  
  "type": "https://example.com/errors/out-of-credit",  
  "title": "You do not have enough credit.",  
  "status": 403,  
  "detail": "Your balance is 30, but that costs 50."  
}
```

(The last is a standard format called **Problem Details**.) Also note any **request ID** header (X-Request-ID, request-id, x-github-request-id). Include it when contacting the provider's support; it lets them find your exact call in their logs.

A simple debugging order for any failing call:

1. **What's the status code?** 4xx → look at your request. 5xx → the provider's problem (retry, check their status page).
2. **What does the error body say?** It usually tells you exactly what's wrong.
3. **Compare against a working example.** Get it working in Postman first, then compare the exact URL, headers, and body your code sends.

Webhooks: API Calls That Come to You

Some things happen on the provider's side *later*: a payment completes, a customer cancels a subscription, someone opens a pull request. You could **poll** ("anything new? anything new?" every minute), which is wasteful and slow.

Webhooks reverse the direction:

1. You build an endpoint in *your* API, e.g. POST `https://yourapp.com/webhooks/stripe`.
2. You register that URL in the provider's dashboard and choose which events you want.
3. When an event happens, the provider sends an HTTP POST to your URL with a JSON body describing it:

```
{  
  "id": "evt_1Q2w3E4r",
```

```

    "type": "payment.succeeded",
    "created": 1790361130,
    "data": { "payment_id": "pay_789", "amount": 4900, "currency": "usd" }
  }

```

4. Your endpoint processes it and returns **2xx** quickly to say “received”.

Webhook rules you must follow:

- **Verify the signature.** Anyone on the internet can POST to your URL. Providers sign each webhook with a shared secret and put the signature in a header (`Stripe-Signature`, `X-Hub-Signature-256`, `X-Signature`). Your code recomputes it and rejects mismatches. You’ll implement this in Chapter 15.
- **Respond fast.** Return 200 within a few seconds and do slow work afterwards (in a background task). Otherwise the provider assumes failure.
- **Expect duplicates.** If your response is slow or lost, the provider **retries**, so you may receive the same event twice. Use the event `id` to ignore events you’ve already processed.
- **Local testing needs a public URL.** The provider can’t reach `localhost`. Tools like **ngrok** or **cloudflared** create a temporary public URL that forwards to your laptop. Many providers also have a CLI that forwards test events to `localhost` (e.g. `stripe listen`).

Webhooks are central to paid integration work: payments, e-commerce, CRMs, form tools, and Git hosting all use them.

Lab: Paging Through a Real API

Using `fetch_all_repos`, count how many public repos the `fastapi` GitHub account has. Then change the function to print the `x-ratelimit-remaining` header after each page. How many calls did it cost? Without a token you get 60 calls an hour. What do you think happens on call 61? (GitHub answers with a 403 or 429 and a message about rate limits. Find out by reading the response body rather than guessing.)

Expected results: One call per page of 100 repos, each lowering `x-ratelimit-remaining` by one. Past the hourly limit, GitHub answers 403 or 429 with a message beginning `API rate limit exceeded`.

Summary, Key Terms, and Review Questions

Summary

- Expect pagination: `offset/limit`, `cursors`, or `Link` headers. Loop until there’s no next page.

- Respect rate limits: watch `X-RateLimit-*` headers, and on 429 wait (`Retry-After`) before retrying.
- Always set timeouts. Retry network errors, 429, and 5xx with exponential backoff. Don't retry other 4xx.
- Use idempotency keys when retrying anything involving money or irreversible actions.
- Read error bodies and note request IDs.
- Webhooks: the provider POSTs events to your URL. Verify signatures, respond fast, handle duplicates.

Key Terms

Pagination, Offset/limit pagination, Cursor pagination, Rate limit, Backoff (exponential), Jitter, Idempotency key, Problem Details, Request ID, Webhook, HMAC, Polling, Tunnel. Each term is defined in Appendix D.

Review Questions

1. Compare offset/limit pagination with cursor pagination.
2. Which failures should you retry, and which should you not?
3. What problem does an idempotency key solve?
4. Name three rules for handling webhooks safely.

Building APIs with FastAPI



What to ignore for now: async database drivers, Alembic migrations in depth, microservices, Kubernetes, GraphQL servers, WebSockets, message queues (Celery, RabbitMQ, Kafka), caching layers (Redis), and “clean architecture” patterns with many layers. They solve problems you don’t have yet. Build the simple version first.

What you’re building. A Bookmarks API:

- Users **register** and **log in**, and receive a token.
- Each user can **create, list, search, view, update, and delete** their own bookmarks.
- When a bookmark is saved, the API **fetches the page’s title** from the web and asks an **AI model for a two-sentence summary**, in the background.
- It has **tests**, and is **packaged with Docker** and deployable.

Each chapter adds one capability to the same codebase. The final project structure (you’ll grow into it gradually) is:

```
bookmarks-api/  
├── .env                      # secrets (never committed)  
├── .gitignore  
├── requirements.txt  
├── Dockerfile  
├── app/  
│   ├── __init__.py  
│   ├── main.py              # creates the app, wires everything together  
│   └── config.py            # settings from environment variables
```

```
| |— database.py      # database connection
| |— models.py       # database tables
| |— schemas.py      # request/response shapes
| |— security.py     # passwords, tokens, current user
| |— enrich.py       # calls other APIs (web page + AI)
| |— routers/
| |   |— __init__.py
| |   |— auth.py      # /auth endpoints
| |   |— bookmarks.py # /bookmarks endpoints
|— tests/
|   |— __init__.py
|   |— conftest.py
|   |— test_bookmarks.py
```

The Python You Need First

FastAPI is built on three pieces of modern Python that many Python users haven't needed before: **type hints**, **Pydantic**, and **async**. You need a working understanding of each, not mastery. This chapter gives you exactly that much.

Type Hints

A **type hint** labels what type a variable, parameter, or return value is supposed to be:

```
def greet(name: str, times: int = 1) -> str:
    return ("Hello, " + name + "! ") * times
```

`name: str` means “name should be a string”. `-> str` means “returns a string”. In plain Python, hints are just labels. Python doesn't enforce them at runtime, and `greet(42)` would still run (and crash inside). They help your editor autocomplete and catch mistakes.

FastAPI is different: it reads your type hints and acts on them. If you write `bookmark_id: int`, FastAPI converts the incoming text “17” from the URL into the integer 17, and rejects “abc” with a clear error. Type hints are how you tell FastAPI what your API accepts.

The hint forms you'll use:

```
count: int = 0
price: float = 9.99
name: str = "Asha"
active: bool = True

tags: list[str] = ["python", "web"]           # a list of strings
scores: dict[str, int] = {"asha": 10}         # dict with str keys, int values

nickname: str | None = None                  # a string OR None ("optional")
```

`str | None` is the most important one here: it means “this may be missing”. Combined

with a default of `None`, it makes a parameter optional. (Older code writes `Optional[str]`, which means the same thing.)

You'll also see **Annotated**, which attaches extra information to a type:

```
from typing import Annotated
limit: Annotated[int, "extra info FastAPI can read"] = 20
```

You'll see it throughout the official FastAPI docs. It's covered in Chapter 13.

Pydantic: Data Validation with Classes

Pydantic lets you describe the *shape* of some data as a class, then checks incoming data against it:

```
from pydantic import BaseModel, Field, HttpUrl

class BookmarkCreate(BaseModel):
    url: HttpUrl
    title: str | None = Field(default=None, max_length=200)
    notes: str | None = None
```

This says: a `BookmarkCreate` has a `url` that must be a valid web address, an optional `title` of at most 200 characters, and an optional `notes` string.

Creating one **validates** the data:

```
good = BookmarkCreate(url="https://fastapi.tiangolo.com", title="FastAPI")
print(good.url)           # https://fastapi.tiangolo.com/
print(good.title)         # FastAPI
print(good.model_dump())  # {'url': HttpUrl('https://fastapi.tiangolo.com/'),
↪   'title': 'FastAPI', 'notes': None}
```

(`HttpUrl` normalises the address, which is why a trailing `/` appeared.)

Bad data raises a `ValidationError` that says exactly what's wrong:

```
BookmarkCreate(url="not a url", title="x" * 300)
# ValidationError: 2 validation errors for BookmarkCreate
# url
#   Input should be a valid URL, relative URL without a base
# title
#   String should have at most 200 characters
```

Pydantic also **converts** compatible data. For an `int` field, the string `"42"` becomes `42`. For a `datetime` field, the string `"2026-09-25T14:30:00Z"` becomes a real `datetime` object. And it goes the other way: `model.model_dump_json()` turns the object into JSON text.

Useful Field options: `min_length`, `max_length` (strings, lists), `gt`, `ge`, `lt`, `le` (numbers: greater than, greater or equal, and so on), and `default`.

Special types you'll use: `HttpUrl`, `EmailStr` (needs the `email-validator` package, included with FastAPI's standard install), and `datetime`.

Why this matters for APIs: every request body arriving at your API is untrusted text from the internet. With Pydantic, you declare the shape once, and all the checking (types, lengths, required fields, formats) happens automatically before your code runs. FastAPI uses Pydantic for every request and response.

Async: Just Enough

Some operations make your program *wait*: calling another API, querying a database, reading a file. During that wait, the processor is idle.

Synchronous code waits and does nothing else:

```
import httpx

def get_two_users():
    a = httpx.get("https://api.github.com/users/octocat")    # wait ~200ms
    b = httpx.get("https://api.github.com/users/torvalds")    # wait ~200ms more
    return a.json(), b.json()
```

Asynchronous code can start other work while waiting:

```
import asyncio
import httpx

async def get_two_users():
    async with httpx.AsyncClient() as client:
        a, b = await asyncio.gather(                        # run both at once
            client.get("https://api.github.com/users/octocat"),
            client.get("https://api.github.com/users/torvalds"),
        )
    return a.json(), b.json()

asyncio.run(get_two_users())
```

The vocabulary:

- `async def` defines a *coroutine function*: a function that can pause while waiting.
- `await` marks the pause points: “wait for this, and let other work run meanwhile”.
- You can only use `await` inside an `async def`.
- An **event loop** runs coroutines, switching between them at `await` points. Your web server runs one for you.

How this affects FastAPI (the only rule you truly need):

- If your endpoint uses **async libraries** (like `httpx.AsyncClient`) and awaits them, declare it `async def`.
- If your endpoint uses **normal (blocking) code**, such as a regular database session, `httpx.get`, or `time.sleep`, declare it with plain `def`. FastAPI then runs it in a separate thread so it doesn’t freeze other requests.
- **Never call slow blocking code inside an `async def` endpoint.** It stalls *every* request to your server while it waits. When in doubt, use plain `def`. It’s always safe.

Your Bookmarks API uses plain `def` almost everywhere, because it uses a regular (synchronous) database session. That’s a perfectly good, common choice. Chapter 15 shows an `async def` endpoint where it genuinely helps.

Lab: Validating a Signup Form

Write a Pydantic model `Signup` with `email: EmailStr`, `password: str` (at least 8 characters), and `age: int` (at least 13). Try creating it with good data, then with a bad email, a short password, and `age="twelve"`. Read each error message.

Expected results: Good data creates the object. A bad email gives `value is not a valid email address`, a short password gives `String should have at least 8 characters`, `age="twelve"` gives `Input should be a valid integer`, and `age=12` gives `Input should be greater than or equal to 13`.

Summary, Key Terms, and Review Questions

Summary

- Type hints (`int`, `str` | `None`, `list[str]`) are labels in Python, but **instructions** in FastAPI.
- Pydantic models declare data shapes and automatically validate and convert data.
- `async def` + `await` let a program do other work while waiting on I/O.
- FastAPI rule: `async def` only when you `await` async libraries; otherwise plain `def`.

Key Terms

Type hint, Pydantic, Validation, Coroutine, `async` / `await`, Event loop. Each term is defined in Appendix D.

Review Questions

1. What does FastAPI do with a type hint like `bookmark_id: int`?
2. What happens when you create a Pydantic model with invalid data?
3. When should a FastAPI endpoint be `async def`, and when plain `def`?
4. Why is calling blocking code inside `async def` harmful?

Your First FastAPI App

In this chapter you install FastAPI, write your first endpoints, and see how FastAPI turns ordinary Python functions and type hints into a working, documented API.

What FastAPI Is (and the Pieces Around It)

FastAPI is a Python framework for building web APIs. You write normal Python functions; FastAPI handles receiving HTTP requests, calling the right function, validating inputs, converting outputs to JSON, and generating documentation.

It sits on top of a few other pieces, which you'll see in error messages and logs:

- **Starlette**: the underlying web toolkit FastAPI is built on (routing, requests, responses).
- **Pydantic**: validation (Chapter 9).
- **Uvicorn**: the **server**, the program that actually listens on a port (Chapter 2), accepts HTTP connections, and hands each request to your FastAPI app.
- **ASGI**: the standard way a Python server (Uvicorn) talks to a Python web app (FastAPI). You don't write ASGI code; just know that "ASGI server" means "something like Uvicorn".

So: **Uvicorn listens on port 8000 → hands the request to FastAPI → FastAPI calls your function → your function returns data → FastAPI turns it into an HTTP response → Uvicorn sends it back.**

Why FastAPI rather than Flask or Django? It's fast to write, it validates automatically, it produces interactive docs for free, it's one of the most-used Python API frameworks, and it's the default choice for AI/ML backends, which is a big share of current paid work.

Setting Up the Project

```
mkdir bookmarks-api
cd bookmarks-api
```

```
python3 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install "fastapi[standard]"
```

A **virtual environment** (`.venv`) is a private folder of packages for this project, so projects don't conflict with each other. Activate it every time you open a new terminal for this project; your prompt shows (`.venv`) when it's active.

`fastapi[standard]` installs FastAPI plus Uvicorn, the `fastapi` command-line tool, and a few extras you'll need later (form parsing, email validation).

Hello, API

Create `main.py`:

```
from fastapi import FastAPI

app = FastAPI(title="Bookmarks API")

@app.get("/")
def home():
    return {"message": "Hello, API!"}
```

Run it:

```
fastapi dev main.py
```

You'll see output ending with something like Uvicorn running on `http://127.0.0.1:8000`. Your API is live on **your computer, port 8000**. Open `http://localhost:8000` in a browser, or:

```
curl -i http://localhost:8000

HTTP/1.1 200 OK
content-type: application/json

{"message": "Hello, API!"}
```

What just happened, line by line:

- `app = FastAPI(...)` creates your application.
- `@app.get("/")` is a **decorator** that registers the function below it as the handler for **GET requests to the path /**. This is **routing**.

- The function returns a Python dict; FastAPI converts it to JSON and sets Content-Type: application/json and status 200 automatically.

`fastapi dev` runs in **development mode** with **auto-reload**: save a change to `main.py` and the server restarts itself. (For production you'll use `fastapi run`, in Chapter 16.) Stop the server with `Ctrl+C`.

Path Parameters

Add:

```
@app.get("/greet/{name}")
def greet(name: str):
    return {"greeting": f"Hello, {name}!"}
```

`{name}` in the path is a **path parameter**. FastAPI matches it to the function argument with the same name. Visit `http://localhost:8000/greet/Asha`.

Types matter:

```
@app.get("/squares/{number}")
def square(number: int):
    return {"number": number, "square": number * number}
```

`/squares/7` returns `{"number": 7, "square": 49}`. Note that `number` arrived as the *text* `"7"` in the URL, and FastAPI converted it to an `int` because of the type hint. Now try `/squares/seven`:

```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["path", "number"],
      "msg": "Input should be a valid integer, unable to parse string as an
        ↪ integer",
      "input": "seven"
    }
  ]
}
```

Status **422**. You wrote no validation code, but FastAPI rejected the bad input with a precise message: where (path → number), what's wrong, and what was received. Your function never ran.

Query Parameters

Any function argument that isn't in the path becomes a query parameter:

```
@app.get("/greet/{name}")
def greet(name: str, excited: bool = False, times: int = 1):
    punctuation = "!" if excited else "."
    return {"greeting": f"Hello, {name}{punctuation} " * times}
```

- /greet/Asha → Hello, Asha.
- /greet/Asha?excited=true×=2 → Hello, Asha! Hello, Asha!

Arguments **with a default** are optional query parameters; **without a default**, they're required (missing → 422). FastAPI converts true, 1, yes, and on to True for bool.

So FastAPI decides where each argument comes from:

Argument	Comes from
Name appears in the path ({name})	Path
Simple type (int, str, bool...) not in path	Query string
Pydantic model	Request body (Chapter 11)

The Free Documentation: /docs

Open <http://localhost:8000/docs>.

This is **Swagger UI**: interactive documentation generated automatically from your code. Every endpoint is listed with its method, path, parameters and types. Click one → **Try it out** → fill in values → **Execute**, and you see the real request (as a curl command), the response, and the status code. It's a built-in Postman for your own API.

There's also /redoc (a read-only documentation style), and /openapi.json: the raw **OpenAPI** document describing your whole API in a standard machine-readable format. /docs is drawn from it, and so can other tools be.

This is a major selling point in client work. Your API documents itself, always matches the code, and clients can try it in a browser.

Using Your API from Postman

1. In Postman, click **Import**, paste <http://localhost:8000/openapi.json>, and import it as a collection. Postman creates a request for every endpoint.
2. Create an environment **Local** with base_url = <http://localhost:8000>, and select it.

3. Make sure the imported requests use `{{base_url}}` (Postman usually creates a collection variable called `baseUrl`; either works).

From now on, test each new endpoint in both `/docs` and Postman. Re-import when you add endpoints, or add requests by hand.

Lab: A Temperature Converter Endpoint

Add an endpoint `GET /convert/{celsius}` that takes a float and returns `{"celsius": ..., "fahrenheit": ...}`, with an optional query parameter `round_to: int = 1` that rounds the result. Test it in `/docs`, in Postman, and with `curl`. Then try invalid input in all three.

Expected results: `/convert/100` returns `{"celsius": 100.0, "fahrenheit": 212.0}`. `/convert/abc` returns 422 with error type `float_parsing`, in `/docs`, Postman, and `curl` alike.

Summary, Key Terms, and Review Questions

Summary

- Uvicorn (the server) listens on a port and passes requests to FastAPI (your app).
- `@app.get("/path")` registers a function for a method and path. Returning a dict produces JSON.
- `{name}` in the path $\rightarrow$ path parameter. Other simple arguments $\rightarrow$ query parameters. Defaults make them optional.
- Type hints drive automatic conversion and validation; bad input gets a 422 with a precise error.
- `/docs` gives interactive documentation for free; `/openapi.json` can be imported into Postman.

Key Terms

FastAPI, Starlette, Uvicorn, ASGI, Virtual environment, Decorator, Routing, Swagger UI, OpenAPI. Each term is defined in Appendix D.

Review Questions

1. What roles do Uvicorn and FastAPI each play in handling a request?
2. How does FastAPI decide whether an argument comes from the path, the query string, or the body?

3. What happens when a path parameter can't be converted to its declared type?
4. What is `/openapi.json`, and what two tools in this chapter use it?

Request Bodies, Validation, and Responses

Now the real API: bookmark CRUD. In this chapter data is kept in a Python dictionary in memory (it disappears when the server restarts). Chapter 12 swaps in a real database.

Receiving a JSON Body

When a function argument is a **Pydantic model**, FastAPI reads it from the **request body**:

```
from pydantic import BaseModel, Field, HttpUrl

class BookmarkCreate(BaseModel):
    url: HttpUrl
    title: str | None = Field(default=None, max_length=200)
    notes: str | None = Field(default=None, max_length=1000)

@app.post("/bookmarks")
def create_bookmark(data: BookmarkCreate):
    return {"received": data}
```

For a POST `/bookmarks` with body `{"url": "https://example.com", "notes": "read later"}`, FastAPI:

1. Checks Content-Type is JSON and parses the body.
2. Validates it against `BookmarkCreate` (valid URL? title short enough?).
3. If invalid → responds **422** with details; your function never runs.
4. If valid → calls your function with data as a proper `BookmarkCreate` object.

Your function can trust data completely. That's the key benefit: **validation happens at the edge, before your code runs**.

Separate Models for Input and Output

A professional API uses **different models for different directions**:

- **Create** model: what the client sends to create something (no `id`, since the server assigns it).
- **Update** model: what the client may change (every field optional, for PATCH).
- **Read** model: what the API sends back (includes `id`, timestamps; excludes anything private).

Why not one model? Because the client mustn't set the `id` or `created_at` itself, and some data must **never** be sent back. The classic example is a user's password hash:

```
class UserCreate(BaseModel):    # what the client sends
    email: str
    password: str

class UserRead(BaseModel):      # what the API returns: no password!
    id: int
    email: str
```

response_model: Controlling What Goes Out

`response_model` tells FastAPI the shape of the response. FastAPI then **filters and validates** whatever your function returns against it: extra fields are dropped, and the docs show the exact response shape. Even if your function accidentally returns the whole user record including the password hash, `response_model=UserRead` guarantees only `id` and `email` leave the server.

Status Codes and Errors

- Set the success status with `status_code`=, e.g. 201 for create and 204 for delete. The `status` module has named constants (`status.HTTP_201_CREATED`) so you don't need to memorise numbers.
- For errors, **raise** `HTTPException`:

```
from fastapi import HTTPException

raise HTTPException(status_code=404, detail="Bookmark not found")
```

FastAPI stops your function and sends a response with that status and the body `{"detail": "Bookmark not found"}`.

Full In-Memory CRUD

Replace `main.py` with:

```
from datetime import datetime, timezone

from fastapi import FastAPI, HTTPException, Query, status
from pydantic import BaseModel, Field, HttpUrl

app = FastAPI(title="Bookmarks API")

class BookmarkCreate(BaseModel):
    url: HttpUrl
    title: str | None = Field(default=None, max_length=200)
    notes: str | None = Field(default=None, max_length=1000)

class BookmarkUpdate(BaseModel):
    title: str | None = Field(default=None, max_length=200)
    notes: str | None = Field(default=None, max_length=1000)

class BookmarkRead(BaseModel):
    id: int
    url: str
    title: str | None
    notes: str | None
    created_at: datetime

bookmarks: dict[int, dict] = {}  # our "database" for now
next_id = 1

@app.post("/bookmarks", response_model=BookmarkRead,
↳ status_code=status.HTTP_201_CREATED)
def create_bookmark(data: BookmarkCreate):
    global next_id
    bookmark = {
        "id": next_id,
        "url": str(data.url),
        "title": data.title,
        "notes": data.notes,
        "created_at": datetime.now(timezone.utc),
    }
    bookmarks[next_id] = bookmark
    next_id += 1
    return bookmark

@app.get("/bookmarks", response_model=list[BookmarkRead])
def list_bookmarks()
```

```

    offset: int = Query(default=0, ge=0),
    limit: int = Query(default=20, ge=1, le=100),
):
    return list(bookmarks.values())[offset : offset + limit]

@app.get("/bookmarks/{bookmark_id}", response_model=BookmarkRead)
def get_bookmark(bookmark_id: int):
    if bookmark_id not in bookmarks:
        raise HTTPException(status_code=404, detail="Bookmark not found")
    return bookmarks[bookmark_id]

@app.patch("/bookmarks/{bookmark_id}", response_model=BookmarkRead)
def update_bookmark(bookmark_id: int, data: BookmarkUpdate):
    if bookmark_id not in bookmarks:
        raise HTTPException(status_code=404, detail="Bookmark not found")
    changes = data.model_dump(exclude_unset=True)
    bookmarks[bookmark_id].update(changes)
    return bookmarks[bookmark_id]

@app.delete("/bookmarks/{bookmark_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_bookmark(bookmark_id: int):
    if bookmark_id not in bookmarks:
        raise HTTPException(status_code=404, detail="Bookmark not found")
    del bookmarks[bookmark_id]

```

Points worth noticing:

- `Query(default=0, ge=0)` adds validation rules to a query parameter: `offset` can't be negative, and `limit` must be between 1 and 100. That's the pagination from Chapter 8, from the server side. Capping `limit` protects your server from someone requesting a million records.
- `response_model=list[BookmarkRead]`: the response is a list of bookmarks.
- **PATCH and `exclude_unset=True`**: `model_dump(exclude_unset=True)` returns **only the fields the client actually sent**. If they send `{"notes": "great"}`, you get `{"notes": "great"}`, not `{"title": None, "notes": "great"}`, so you don't accidentally wipe the title. This single line is how PATCH semantics are done right.
- `str(data.url): HttpUrl` is a special type; we store it as plain text.
- **DELETE returns nothing**, with 204.
- Datetimes are converted to ISO 8601 strings in the JSON automatically.

Trying the Endpoints in Postman

In Postman (with your Local environment):

1. POST `{{base_url}}/bookmarks`, body (raw JSON): `{"url": "https://fastapi.tiangolo.com", "title": "FastAPI docs"}` → **201**.
2. POST with `{"url": "nope"}` → **422**. Read the error.
3. GET `{{base_url}}/bookmarks` → **200**, a list.
4. PATCH `{{base_url}}/bookmarks/1` with `{"notes": "Start here"}` → the title is unchanged.
5. DELETE `{{base_url}}/bookmarks/1` → **204**. Then GET `/bookmarks/1` → **404**.

Add a Post-response test to each checking the expected status code. You now have a regression suite you can re-run after every change.

A Few More Request Features (for Reference)

You'll need these sometimes; you don't need them for this project:

```
from fastapi import Header, UploadFile, File, Form

@app.get("/whoami")
def whoami(user_agent: str | None = Header(default=None)):  # reads the
    ↪ User-Agent header
    return {"user_agent": user_agent}

@app.post("/upload")
async def upload(file: UploadFile = File(...)):  # multipart file
    ↪ upload
    content = await file.read()
    return {"filename": file.filename, "size": len(content)}

@app.post("/contact")
def contact(name: str = Form(...), message: str = Form(...)):  # HTML form fields
    return {"ok": True}
```

Header converts underscores to dashes, so `user_agent` reads the User-Agent header, and `x_api_key` would read X-API-Key.

Lab: Search and Count Endpoints

Add a `q: str | None = None` query parameter to `list_bookmarks` that filters bookmarks whose title contains `q` (case-insensitive). Then add a `GET /bookmarks/count` endpoint returning `{"count": n}`. Careful: it must be defined **above** `/bookmarks/{bookmark_id}` in the file, or FastAPI will try to treat "count" as a bookmark ID. Routes are matched in order. Try it both ways and see the 422.

Expected results: `/bookmarks?q=fast` returns only bookmarks whose titles contain “fast” in any case. `/bookmarks/count` returns `{"count": 2}` (or however many you created). Defined *below* `/bookmarks/{bookmark_id}`, the same URL returns 422, because count is not an integer.

Summary, Key Terms, and Review Questions

Summary

- A Pydantic-model argument = the JSON request body, validated before your function runs.
- Use separate **Create**, **Update**, and **Read** models.
- `response_model` filters and documents what goes out; it’s your safety net against leaking data.
- Raise `HTTPException(status_code=..., detail=...)` for errors; set `status_code=` on the decorator for success codes.
- PATCH: `model_dump(exclude_unset=True)` gives only the fields the client sent.

Key Terms

Body, **response_model**, **Validation**, **Status code**, **Resource**. Each term is defined in Appendix D.

Review Questions

1. Why use separate Create, Update, and Read models?
2. How does `response_model` protect you from leaking data?
3. What does `model_dump(exclude_unset=True)` do, and why does PATCH need it?
4. Why must `/bookmarks/count` be defined before `/bookmarks/{bookmark_id}`?

Adding a Database

In-memory data vanishes on restart. Real APIs store data in a **database**.

Database Basics in Two Minutes

A **relational database** stores data in **tables** (like spreadsheets): each **row** is one record, each **column** is one field. Every row has a unique **primary key** (usually an auto-incrementing `id`). A **foreign key** is a column pointing to another table's primary key, for example `bookmark.owner_id → user.id`. You talk to it in **SQL** (`SELECT * FROM bookmark WHERE id = 17`).

Which database?

- **SQLite**: the whole database is one file on disk. Zero setup, built into Python. Perfect for learning and small projects.
- **PostgreSQL** (“Postgres”): the standard production database. Runs as a separate server. Use it when you deploy (Chapter 16).

The code below works with both. Only the connection URL changes.

An ORM: SQLAlchemy

Instead of writing SQL strings, you'll use an **ORM** (Object-Relational Mapper): you define tables as Python classes and work with rows as Python objects. The ORM writes the SQL.

You'll use **SQLModel**, created by FastAPI's author. It's built on **SQLAlchemy** (the most widely used Python ORM, which you'll also see in job listings) and on Pydantic, so its classes feel like the Pydantic models you already know.

```
pip install sqlmodel
```

Growing into Multiple Files

One file is getting crowded. Create this layout (move `main.py` into `app/`):

```
bookmarks-api/
├── app/
│   ├── __init__.py      # empty file: makes "app" a Python package
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   └── main.py
```

Run it from the `bookmarks-api` folder with `fastapi dev app/main.py`.

database.py: The Connection

```
from sqlmodel import Session, SQLModel, create_engine

DATABASE_URL = "sqlite:///./bookmarks.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})

def create_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session
```

- The **engine** manages connections to the database. Create it once, at startup. `sqlite:///./bookmarks.db` means “a SQLite file named `bookmarks.db` in the current folder”. (`check_same_thread=False` is a SQLite-only setting needed because FastAPI may use different threads.)
- `create_tables()` creates any tables that don’t exist yet.
- A **session** is one unit of conversation with the database: you add, change, and query objects through it, then **commit** to save. `get_session()` gives out a session and closes it afterwards. The `yield` makes that “afterwards” happen automatically, as you’ll see in a moment.

models.py: The Tables

```
from datetime import datetime, timezone
```

```

from sqlmodel import Field, SQLModel

class Bookmark(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    url: str
    title: str | None = None
    notes: str | None = None
    summary: str | None = None
    created_at: datetime = Field(default_factory=lambda:
    ↪ datetime.now(timezone.utc))

```

`table=True` makes this class a database table named `bookmark`. `id` is `None` until the database assigns one on insert. `default_factory` calls a function to produce each new row's timestamp. (summary will be filled in by AI in Chapter 15.)

schemas.py: The API Shapes

Move your Pydantic models from Chapter 11 here, and add summary to the read model:

```

from datetime import datetime

from pydantic import BaseModel, Field, HttpUrl

class BookmarkCreate(BaseModel):
    url: HttpUrl
    title: str | None = Field(default=None, max_length=200)
    notes: str | None = Field(default=None, max_length=1000)

class BookmarkUpdate(BaseModel):
    title: str | None = Field(default=None, max_length=200)
    notes: str | None = Field(default=None, max_length=1000)

class BookmarkRead(BaseModel):
    id: int
    url: str
    title: str | None
    notes: str | None
    summary: str | None
    created_at: datetime

```

Why keep database models (`models.py`) and API schemas (`schemas.py`) separate? Because they change for different reasons. The database will soon hold things the API must never expose (password hashes, owner IDs, internal flags), and the API needs input shapes

that aren't tables at all. Keeping them apart means a database change can't silently change your public API, and vice versa. It's a small cost now that saves real pain later.

Dependencies: How the Endpoint Gets a Session

Here's the new piece of FastAPI:

```
from fastapi import Depends
from sqlmodel import Session

from app.database import get_session

@app.get("/bookmarks/{bookmark_id}")
def get_bookmark(bookmark_id: int, session: Session = Depends(get_session)):
    ...
```

`Depends(get_session)` tells FastAPI: **“before calling this endpoint, call `get_session()` and pass me what it yields.”** For every request, FastAPI:

1. Runs `get_session()` up to `yield`, which opens a session.
2. Calls your endpoint with that session.
3. After the response is ready, finishes `get_session()` after the `yield`, which closes the session.

This is **dependency injection**: your function declares what it needs, and FastAPI provides it. It's FastAPI's most important feature beyond routing. You'll use the same mechanism for settings, the current logged-in user, and swapping in a test database. Chapter 13 covers it properly.

main.py: CRUD Against the Database

```
from contextlib import asynccontextmanager

from fastapi import Depends, FastAPI, HTTPException, Query, status
from sqlmodel import Session, select

from app.database import create_tables, get_session
from app.models import Bookmark
from app.schemas import BookmarkCreate, BookmarkRead, BookmarkUpdate

@asynccontextmanager
async def lifespan(app: FastAPI):
```

```

    create_tables()          # runs once, at startup
    yield                   # the app runs here

app = FastAPI(title="Bookmarks API", lifespan=lifespan)

@app.post("/bookmarks", response_model=BookmarkRead,
↳ status_code=status.HTTP_201_CREATED)
def create_bookmark(data: BookmarkCreate, session: Session =
↳ Depends(get_session)):
    bookmark = Bookmark(url=str(data.url), title=data.title, notes=data.notes)
    session.add(bookmark)
    session.commit()
    session.refresh(bookmark)      # reload it to get the id the database
↳ assigned
    return bookmark

@app.get("/bookmarks", response_model=list[BookmarkRead])
def list_bookmarks(
    q: str | None = None,
    offset: int = Query(default=0, ge=0),
    limit: int = Query(default=20, ge=1, le=100),
    session: Session = Depends(get_session),
):
    query = select(Bookmark)
    if q:
        query = query.where(Bookmark.title.contains(q))
    query = query.order_by(Bookmark.created_at.desc()).offset(offset).limit(limit)
    return session.exec(query).all()

@app.get("/bookmarks/{bookmark_id}", response_model=BookmarkRead)
def get_bookmark(bookmark_id: int, session: Session = Depends(get_session)):
    bookmark = session.get(Bookmark, bookmark_id)
    if bookmark is None:
        raise HTTPException(status_code=404, detail="Bookmark not found")
    return bookmark

@app.patch("/bookmarks/{bookmark_id}", response_model=BookmarkRead)
def update_bookmark(
    bookmark_id: int, data: BookmarkUpdate, session: Session =
↳ Depends(get_session)
):
    bookmark = session.get(Bookmark, bookmark_id)
    if bookmark is None:
        raise HTTPException(status_code=404, detail="Bookmark not found")
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(bookmark, field, value)
    session.add(bookmark)

```

```

    session.commit()
    session.refresh(bookmark)
    return bookmark

@app.delete("/bookmarks/{bookmark_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_bookmark(bookmark_id: int, session: Session = Depends(get_session)):
    bookmark = session.get(Bookmark, bookmark_id)
    if bookmark is None:
        raise HTTPException(status_code=404, detail="Bookmark not found")
    session.delete(bookmark)
    session.commit()

```

The database operations you need, all shown above:

Goal	Code
Insert	<code>session.add(obj) → session.commit() → session.refresh(obj)</code>
Get one by primary key	<code>session.get(Bookmark, 17)</code> (returns None if missing)
Query many	<code>session.exec(select(Bookmark).where(...).order_by(...).offset(...).limit(...)).all()</code>
First match	<code>session.exec(select(...).where(...)).first()</code>
Update	<code>change attributes → session.add(obj) → session.commit()</code>
Delete	<code>session.delete(obj) → session.commit()</code>

The **lifespan** function runs setup code when the server starts (and cleanup after yield, when it stops). It's the right place to create tables or open shared connections.

Notice the endpoints **return Bookmark database objects**, but `response_model=BookmarkRead` converts them into the public shape. The separation of models and schemas working as designed.

Run `fastapi dev app/main.py`, create a few bookmarks, then **stop and restart the server**. They're still there, in `bookmarks.db`. (VS Code extensions such as "SQLite Viewer" let you open the file and see the rows.)

Changing Tables Later: Migrations



`create_all` only creates **missing tables**. It never changes an existing table. If you add a column to `Bookmark` later, the existing `bookmarks.db` won't get it. While learning, the fix is simple: delete `bookmarks.db` and restart. You'll do this in Chapter 14.

With real data you can't delete the database. Production projects use **migrations**, versioned scripts that alter tables step by step, with a tool called **Alembic**. Learn it when you have your first real deployment with data you care about; the Alembic tutorial takes an afternoon.

Lab: Adding a Favourites Column

Add a `is_favourite: bool = False` column to `Bookmark` (and to `BookmarkRead` and `BookmarkUpdate`). Restart, and notice the error or missing column until you delete `bookmarks.db`. Then add a `favourites_only: bool = False` query parameter to `list_bookmarks`.

Expected results: Before you delete `bookmarks.db`, bookmark requests fail with 500 Internal Server Error and the server log shows `no such column: bookmark.is_favourite`. After deleting it and restarting, `?favourites_only=true` returns only bookmarks you've marked with `PATCH {"is_favourite": true}`.

Summary, Key Terms, and Review Questions

Summary

- Tables are `SQLModel` classes with `table=True`; API shapes stay in separate Pydantic schemas.
- The engine is created once; each request gets its own session via `Depends(get_session)`.
- `add + commit` saves; `session.get` fetches by ID; `select(...)` builds queries.
- `lifespan` runs startup and shutdown code.
- `create_all` doesn't alter existing tables; use migrations (Alembic) once real data exists.

Key Terms

Relational database, Table (database), Primary key, Foreign key, SQLite / PostgreSQL, ORM, `SQLModel` / `SQLAlchemy`, Engine (database), Session (database), Commit (database), Dependency (FastAPI), Dependency injection, Lifespan, Migration, Alembic. Each term is defined in Appendix D.

Review Questions

1. What is a database session, and why does each request get its own?
2. Why keep database models separate from API schemas?
3. What does `Depends(get_session)` do, step by step?

4. Why doesn't `create_all` add a new column, and what do production projects use instead?

Structuring a Real Project

Before adding logins, give the project a shape that will hold up as it grows: **settings** from the environment, **routers** to split endpoints across files, and a proper understanding of **dependencies**.

Settings from Environment Variables

Hard-coded values like the database URL must change between your laptop and production, and secrets must never be in code (Chapter 7). The standard FastAPI solution is **pydantic-settings**: a Pydantic model whose fields are filled from environment variables or a `.env` file.

```
pip install pydantic-settings
```

app/config.py:

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env")

    database_url: str = "sqlite:///./bookmarks.db"
    jwt_secret: str
    jwt_algorithm: str = "HS256"
    access_token_minutes: int = 60
    anthropic_api_key: str = ""
    cors_origins: list[str] = ["http://localhost:3000"]

settings = Settings()
```

Each field is read from the environment variable of the same name (case-insensitive: `DATABASE_URL` fills `database_url`), falling back to `.env`, then to the default. Fields **without**

a default (`jwt_secret`) are **required**: the app refuses to start without them, with a clear error. That's what you want: fail loudly at startup, not mysteriously later. Values are validated and converted too (`ACCESS_TOKEN_MINUTES=30` becomes an `int`).

Create `.env` in the project root (`bookmarks-api/`):

```
JWT_SECRET=change-me-to-a-long-random-string
ANTHROPIC_API_KEY=
```

Generate a proper random secret with `python3 -c "import secrets; print(secrets.token_hex(32))"`. `JWT_SECRET` is used in Chapter 14, and `ANTHROPIC_API_KEY` in Chapter 15.

Create `.gitignore`:

```
.venv/
__pycache__/
.env
*.db
```

And update `app/database.py` to use settings:

```
from sqlalchemy import Session, SQLModel, create_engine

from app.config import settings

connect_args = {"check_same_thread": False} if
↳ settings.database_url.startswith("sqlite") else {}
engine = create_engine(settings.database_url, connect_args=connect_args)

def create_tables():
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session
```

Now switching to Postgres in production is just setting `DATABASE_URL`. No code change.

Routers: Splitting Endpoints Across Files

An `APIRouter` is a mini-app holding a group of related endpoints. You create one per area of your API and plug them all into the main app.

Create `app/routers/__init__.py` (empty) and `app/routers/bookmarks.py`. Move the bookmark endpoints there, changing `@app.` to `@router.` and removing `/bookmarks` from each path (the router's prefix adds it). We'll also pull the repeated "fetch or 404" code into a dependency:

```
from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlmodel import Session, select

from app.database import get_session
from app.models import Bookmark
from app.schemas import BookmarkCreate, BookmarkRead, BookmarkUpdate

router = APIRouter(prefix="/bookmarks", tags=["bookmarks"])

def get_bookmark_or_404(bookmark_id: int, session: Session = Depends(get_session))
↳ -> Bookmark:
    bookmark = session.get(Bookmark, bookmark_id)
    if bookmark is None:
        raise HTTPException(status_code=404, detail="Bookmark not found")
    return bookmark

@router.post("", response_model=BookmarkRead, status_code=status.HTTP_201_CREATED)
def create_bookmark(data: BookmarkCreate, session: Session =
↳ Depends(get_session)):
    bookmark = Bookmark(url=str(data.url), title=data.title, notes=data.notes)
    session.add(bookmark)
    session.commit()
    session.refresh(bookmark)
    return bookmark

@router.get("", response_model=list[BookmarkRead])
def list_bookmarks(
    q: str | None = None,
    offset: int = Query(default=0, ge=0),
    limit: int = Query(default=20, ge=1, le=100),
    session: Session = Depends(get_session),
):
    query = select(Bookmark)
    if q:
        query = query.where(Bookmark.title.contains(q))
    query = query.order_by(Bookmark.created_at.desc()).offset(offset).limit(limit)
    return session.exec(query).all()

@router.get("/{bookmark_id}", response_model=BookmarkRead)
def get_bookmark(bookmark: Bookmark = Depends(get_bookmark_or_404)):
    return bookmark
```

```

@router.patch("/{bookmark_id}", response_model=BookmarkRead)
def update_bookmark(
    data: BookmarkUpdate,
    bookmark: Bookmark = Depends(get_bookmark_or_404),
    session: Session = Depends(get_session),
):
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(bookmark, field, value)
    session.add(bookmark)
    session.commit()
    session.refresh(bookmark)
    return bookmark

@router.delete("/{bookmark_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_bookmark(
    bookmark: Bookmark = Depends(get_bookmark_or_404),
    session: Session = Depends(get_session),
):
    session.delete(bookmark)
    session.commit()

```

`app/main.py` becomes small. It just assembles the pieces:

```

from contextlib import asynccontextmanager

from fastapi import FastAPI

from app.database import create_tables
from app.routers import bookmarks

@asynccontextmanager
async def lifespan(app: FastAPI):
    create_tables()
    yield

app = FastAPI(title="Bookmarks API", version="1.0.0", lifespan=lifespan)
app.include_router(bookmarks.router)

@app.get("/health")
def health():
    return {"status": "ok"}

```

`tags=["bookmarks"]` groups these endpoints under a heading in `/docs`. The `/health` endpoint is a convention: hosting platforms and monitoring tools call it to check the app is alive (Chapter 16).

Dependencies, Properly

Look at `get_bookmark_or_404`. It's a dependency that **itself depends** on `bookmark_id` (from the path) and on `get_session`. FastAPI resolves the whole chain automatically:

```

update_bookmark
├── data           ← request body (BookmarkUpdate)
├── bookmark       ← get_bookmark_or_404
│   ├── bookmark_id ← path
│   └── session     ← get_session
└── session         ← get_session (the same session is reused within one request)

```

Things to know about dependencies:

- A dependency is **any function** (or class). Its parameters are resolved exactly like an endpoint's: path, query, body, headers, or other dependencies.
- If a dependency **raises** `HTTPException`, the request stops there. That's how `get_bookmark_or_404` returns 404 without the endpoint containing any error code, and how authentication will reject requests in Chapter 14.
- Within one request, each dependency runs **once** and its result is shared. Both sessions above are the same object.
- Dependencies with `yield` run cleanup code after the response (closing sessions, releasing resources).
- They can be **replaced in tests** (Chapter 16). This is a big reason to use them.

Use dependencies for anything several endpoints need: database sessions, the current user, permission checks, shared parameters (like pagination), and clients for other APIs.

The Annotated Style

The FastAPI docs write dependencies and parameter rules in a newer style using `Annotated`:

```

from typing import Annotated

SessionDep = Annotated[Session, Depends(get_session)]

@router.get("")
def list_bookmarks(session: SessionDep, limit: Annotated[int, Query(le=100)] =
    ↳ 20):
    ...

```

It means exactly the same as `session: Session = Depends(get_session)`, and lets you define a reusable alias like `SessionDep`. This book uses the default-value style because it's easier to read when you're starting out. Recognise both, and pick one per project.

Recording Your Dependencies

Create `requirements.txt` so anyone (including your future server) can install the exact same packages:

```
fastapi[standard]==0.141.1
sqlmodel==0.0.47
pydantic-settings==2.15.0
pyjwt==2.15.0
pwnlib[argon2]==0.3.1
httpx==0.28.1
anthropic==1.8.0
pytest==9.1.1
```

(These are the versions this book was tested with. The last five are installed in the coming chapters. `pip install -r requirements.txt` installs everything. When you start your own project, `pip freeze` shows what's installed in your environment.)

Lab: A Stats Router

Add a `GET /stats` endpoint in a new router file `app/routers/stats.py` that returns the total number of bookmarks. Wire it into `main.py`. Check that it appears under its own heading in `/docs`.

Expected results: `GET /stats` returns a count such as `{"total": 3}`, and `/docs` shows the endpoint under its own **stats** heading (give the router `tags=["stats"]`).

Summary, Key Terms, and Review Questions

Summary

- `pydantic-settings` reads configuration from environment variables and `.env`; required settings without defaults fail fast at startup.
- `.env` and `*.db` go in `.gitignore`.
- `APIRouter(prefix=..., tags=...)` groups endpoints in their own file; `app.include_router()` plugs them in.
- Dependencies can depend on other dependencies, can stop a request by raising `HTTPException`, run once per request, and can be swapped in tests.

Key Terms

Settings (pydantic-settings), Environment variable, APIRouter, Dependency (FastAPI), Requirements file. Each term is defined in Appendix D.

Review Questions

1. Why should `jwt_secret` have no default value?
2. What does an `APIRouter`'s `prefix` do?
3. What happens when a dependency raises `HTTPException`?
4. Why do dependencies make testing easier?

Securing Your API

Right now anyone can read and delete everyone's bookmarks. This chapter adds **users**, **login with JWT tokens**, and **ownership** so each user sees only their own bookmarks.

```
pip install pyjwt "pwplib[argon2]"
```

Never Store Passwords, Store Hashes

You must **never** store users' passwords as plain text. If your database leaks, every user's password leaks, and people reuse passwords everywhere.

Instead store a **hash**: the output of a one-way function. `hash("secret123")` gives something like `$argon2id$v=19$m=65536...`, and there is no way to reverse it. At login you hash the password they typed and compare the hashes. Password-hashing algorithms like **Argon2** are deliberately slow and include a random **salt**, which makes mass guessing impractical. The `pwplib` library handles all of this.

The User Table and Schemas

Add to `app/models.py`, and add an `owner_id` to `Bookmark`:

```
class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    email: str = Field(unique=True, index=True)
    hashed_password: str

class Bookmark(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    url: str
    title: str | None = None
    notes: str | None = None
    summary: str | None = None
```

```

    created_at: datetime = Field(default_factory=lambda:
↳    datetime.now(timezone.utc))
    owner_id: int = Field(foreign_key="user.id", index=True)

```

`owner_id` is a **foreign key** to the user who owns the bookmark. `index=True` makes lookups by that column fast. Because the bookmark table changed, **delete bookmarks.db** before restarting (Chapter 12).

Add to `app/schemas.py` (and add `EmailStr` to the `pydantic` import line):

```

from pydantic import BaseModel, EmailStr, Field, HttpUrl

class UserCreate(BaseModel):
    email: EmailStr
    password: str = Field(min_length=8)

class UserRead(BaseModel):
    id: int
    email: str

class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"

```

Note that `UserRead` has no password field, so the hash can never leave the API.

security.py: Hashing, Tokens, and “Who Is Calling?”

`app/security.py`:

```

from datetime import datetime, timedelta, timezone

import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from pwdlib import PasswordHash
from sqlmodel import Session

from app.config import settings
from app.database import get_session
from app.models import User

password_hash = PasswordHash.recommended()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/token")

```

```

def hash_password(password: str) -> str:
    return password_hash.hash(password)

def verify_password(password: str, hashed: str) -> bool:
    return password_hash.verify(password, hashed)

def create_access_token(user_id: int) -> str:
    expires = datetime.now(timezone.utc) +
    ↪ timedelta(minutes=settings.access_token_minutes)
    payload = {"sub": str(user_id), "exp": expires}
    return jwt.encode(payload, settings.jwt_secret,
    ↪ algorithm=settings.jwt_algorithm)

def get_current_user(
    token: str = Depends(oauth2_scheme),
    session: Session = Depends(get_session),
) -> User:
    credentials_error = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid or expired token",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, settings.jwt_secret,
    ↪ algorithms=[settings.jwt_algorithm])
        user_id = int(payload["sub"])
    except (jwt.InvalidTokenError, KeyError, ValueError):
        raise credentials_error
    user = session.get(User, user_id)
    if user is None:
        raise credentials_error
    return user

```

How it fits together:

- `create_access_token` builds a JWT (Chapter 7) whose payload says “this is user 42, valid until this time”, signed with `JWT_SECRET`. Anyone can *read* the payload; only your server can *create a valid signature*.
- `oauth2_scheme` is a dependency that reads the `Authorization: Bearer <token>` header and returns the token. If the header is missing, it responds **401** on its own. `tokenUrl="auth/token"` tells /docs where to log in.
- `get_current_user` is the key dependency: it takes the token, **verifies the signature and expiry** (`jwt.decode` raises `InvalidTokenError`, which includes `ExpiredSignatureError`, if anything is wrong), loads the user, and returns it, or rejects the request with 401. Any endpoint that declares `user: User = Depends(get_current_user)` is now **protected**.

The Auth Router: Register and Log In

app/routers/auth.py:

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlmodel import Session, select

from app.database import get_session
from app.models import User
from app.schemas import Token, UserCreate, UserRead
from app.security import create_access_token, hash_password, verify_password

router = APIRouter(prefix="/auth", tags=["auth"])

@router.post("/register", response_model=UserRead,
↳ status_code=status.HTTP_201_CREATED)
def register(data: UserCreate, session: Session = Depends(get_session)):
    existing = session.exec(select(User).where(User.email == data.email)).first()
    if existing:
        raise HTTPException(status_code=409, detail="Email already registered")
    user = User(email=data.email, hashed_password=hash_password(data.password))
    session.add(user)
    session.commit()
    session.refresh(user)
    return user

@router.post("/token", response_model=Token)
def login(
    form: OAuth2PasswordRequestForm = Depends(),
    session: Session = Depends(get_session),
):
    user = session.exec(select(User).where(User.email == form.username)).first()
    if user is None or not verify_password(form.password, user.hashed_password):
        raise HTTPException(status_code=401, detail="Wrong email or password")
    return Token(access_token=create_access_token(user.id))
```

- **Register** returns **409 Conflict** for a duplicate email, and `UserRead` so no hash is returned.
- **Login** uses `OAuth2PasswordRequestForm`, which reads **form data** (username and password fields, application/x-www-form-urlencoded), because that's what the OAuth2 "password" standard specifies and what /docs sends. The "username" here is the email.
- The error is deliberately vague ("wrong email or password"), so attackers can't discover which emails are registered.

Protecting Bookmarks and Enforcing Ownership

Update `app/routers/bookmarks.py`. Replace `get_bookmark_or_404` with an ownership-aware version, and add the current user to every endpoint:

```
from app.models import Bookmark, User
from app.security import get_current_user

def get_owned_bookmark(
    bookmark_id: int,
    user: User = Depends(get_current_user),
    session: Session = Depends(get_session),
) -> Bookmark:
    bookmark = session.get(Bookmark, bookmark_id)
    if bookmark is None or bookmark.owner_id != user.id:
        raise HTTPException(status_code=404, detail="Bookmark not found")
    return bookmark

@router.post("", response_model=BookmarkRead, status_code=status.HTTP_201_CREATED)
def create_bookmark(
    data: BookmarkCreate,
    user: User = Depends(get_current_user),
    session: Session = Depends(get_session),
):
    bookmark = Bookmark(url=str(data.url), title=data.title, notes=data.notes,
    ↪ owner_id=user.id)
    session.add(bookmark)
    session.commit()
    session.refresh(bookmark)
    return bookmark

@router.get("", response_model=list[BookmarkRead])
def list_bookmarks(
    q: str | None = None,
    offset: int = Query(default=0, ge=0),
    limit: int = Query(default=20, ge=1, le=100),
    user: User = Depends(get_current_user),
    session: Session = Depends(get_session),
):
    query = select(Bookmark).where(Bookmark.owner_id == user.id)
    if q:
        query = query.where(Bookmark.title.contains(q))
    query = query.order_by(Bookmark.created_at.desc()).offset(offset).limit(limit)
    return session.exec(query).all()
```

The `get/patch/delete` endpoints stay as they were, but depend on `get_owned_bookmark`

instead of `get_bookmark_or_404`. Since `get_owned_bookmark` depends on `get_current_user`, they're protected automatically.

Two important security decisions here:

1. **Every query is filtered by the owner.** Checking “is the user logged in?” is not enough. The most common serious API vulnerability in the real world (it tops the OWASP API Security list, as “Broken Object Level Authorization”) is an API that checks login but lets user A fetch `/bookmarks/17` belonging to user B just by changing the number. Always check that the object belongs to the caller.
2. **Someone else's bookmark returns 404, not 403.** A 403 would confirm that bookmark 17 exists. A 404 reveals nothing.

Finally, register the auth router in `app/main.py`:

```
from app.routers import auth, bookmarks
...
app.include_router(auth.router)
app.include_router(bookmarks.router)
```

Trying It in /docs

Delete `bookmarks.db`, restart, open `/docs`:

1. POST `/auth/register` with `{"email": "asha@example.com", "password": "secret123"}` → 201.
2. Try GET `/bookmarks` → **401**.
3. Click the **Authorize** button (top right), enter the email as username and the password, and click Authorize. `/docs` calls `/auth/token` for you and attaches the token to every following request.
4. GET `/bookmarks` → 200. Create a few bookmarks.
5. Register a second user, authorize as them, and try to GET the first user's bookmark ID → **404**.

Trying It in Postman, with Automatic Token Handling

1. Request **Login**: POST `{{base_url}}/auth/token`, Body → **x-www-form-urlencoded**, keys username and password.
2. Its **Post-response** script:

```
pm.test("Logged in", () => pm.response.to.have.status(200));
pm.environment.set("token", pm.response.json().access_token);
```

3. On the **collection's** Authorization tab: type **Bearer Token**, token `{{token}}`. Set each bookmark request to **Inherit auth from parent**.

Send Login once; every other request is now authenticated. When the token expires (60 minutes), send Login again.

API Keys: The Alternative for Machine Clients

When your callers are **other programs** (a client's backend, a Zapier integration) rather than people logging in, API keys are simpler than logins. A minimal version:

```
from fastapi import Depends, HTTPException
from fastapi.security import APIKeyHeader

api_key_header = APIKeyHeader(name="X-API-Key")

def require_api_key(key: str = Depends(api_key_header)) -> str:
    if key not in VALID_KEYS:          # in real code: look up a hashed key in the
        ↪ database
        raise HTTPException(status_code=401, detail="Invalid API key")
    return key

@app.get("/reports", dependencies=[Depends(require_api_key)])
def reports():
    ...
```

`dependencies=[...]` runs a dependency for its checks only, when you don't need its return value. In production, generate keys with `secrets.token_urlsafe(32)`, show the key to the user once, and store only its hash, exactly like passwords.

CORS: Why Browsers Block Your API (and How to Allow It)

When you (or a client) build a web frontend at `http://localhost:3000` that calls your API at `http://localhost:8000`, the browser will block it with an error mentioning **CORS**. This confuses everyone the first time.

CORS (Cross-Origin Resource Sharing) is a **browser** security rule: JavaScript on one **origin** (scheme + host + port, e.g. `http://localhost:3000`) may not read responses from a different origin unless that server explicitly says it's allowed. It protects users from malicious websites calling other sites' APIs with the user's cookies.

Key points:

- It's enforced **only by browsers**. curl, Postman, Python, and mobile apps ignore it. That's why "it works in Postman but not in the browser".
- The fix is on the **server**: send headers saying which origins are allowed. The browser sometimes sends a **preflight** OPTIONS request first to ask.

In `app/main.py`:

```
from fastapi.middleware.cors import CORSMiddleware

from app.config import settings

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,      # e.g. ["http://localhost:3000",
    ↪ "https://myapp.com"]
    allow_methods=["*"],
    allow_headers=["*"],
)
```

List specific origins. `allow_origins=["*"]` (anyone) is fine for a truly public, read-only API, but not for one with user accounts.

Middleware is code that wraps *every* request and response: it runs before your routes and after them. CORS is the one you'll always need. Others handle compression, logging, or request IDs.

A Security Checklist for Every API You Build

- HTTPS only in production (your hosting platform provides it).
- Hash passwords (Argon2 via `pwdlib`); never log passwords or tokens.
- Validate all input with Pydantic models and limits (`max_length`, `le=100`).
- Check **ownership** on every object access, not just login.
- Return generic errors for auth failures; 404 for other people's objects.
- Secrets in environment variables; long random `JWT_SECRET`; short token lifetimes.
- Specific CORS origins.
- Add **rate limiting** to login and expensive endpoints before going public. It's often done at the hosting or gateway level, or with a library such as `slowapi`.

Lab: A "Who Am I?" Endpoint

Add `GET /auth/me` that returns the current user (`response_model=UserRead`). Then decode one of your tokens at jwt.io: find `sub` and `exp`. Change one character in the token and

call `/auth/me` with it. You should get a 401.

Expected results: `/auth/me` returns `{"id": 1, "email": "asha@example.com"}`. `jwt.io` shows a payload like `{"sub": "1", "exp": 1790364930}`. Change a character in the middle of the token and `/auth/me` returns 401 with `Invalid or expired token`.

Summary, Key Terms, and Review Questions

Summary

- Store password **hashes** (Argon2), never passwords.
- Login returns a signed JWT; `get_current_user` verifies it on each request and loads the user.
- Protect an endpoint by depending on `get_current_user`; enforce **ownership** in every query.
- API keys suit machine clients: read with `APIKeyHeader`, store hashed.
- CORS is a browser rule; allow your frontend's origin with `CORSMiddleware`.

Key Terms

Hash (password), Salt, Argon2, BOLA (Broken Object Level Authorization), CORS, Origin, Preflight request, Middleware, Rate limit. Each term is defined in Appendix D.

Review Questions

1. Why store password hashes instead of passwords?
2. What does `get_current_user` check before it returns a user?
3. Why isn't "is the user logged in?" enough? What else must every endpoint check?
4. Why return 404 rather than 403 for another user's bookmark?
5. Why can a call work in Postman but fail in the browser with a CORS error?

Calling Other APIs from FastAPI

Your API can now be *called*. Much paid work is about your API *calling others*: fetching data, charging cards, sending emails, and calling AI models. This chapter makes the Bookmarks API fetch each page’s title and generate an AI summary, and shows how to receive webhooks.

The Plan

When a bookmark is created:

1. Save it and **respond immediately** (201). The user shouldn’t wait for slow external calls.
2. **In the background**: download the web page, extract its <title>, send its text to an AI model for a two-sentence summary, and save both.

Background Tasks

FastAPI’s `BackgroundTasks` runs a function **after the response has been sent**:

```
from fastapi import BackgroundTasks

@router.post("")
def create_bookmark(data: BookmarkCreate, background_tasks: BackgroundTasks, ...):
    ...
    background_tasks.add_task(enrich_bookmark, bookmark.id)  # runs after
    ↪ responding
    return bookmark
```

This is ideal for short follow-up work: sending a welcome email, calling a webhook, enriching a record. (For heavy or long jobs that must survive a server restart, teams use a **task queue** like Celery, RQ, or ARQ. Not needed yet.)

Calling an AI Model: The Anthropic SDK

For the summary you'll call Anthropic's Claude API. You already saw the raw HTTP call with curl in Chapter 7. In Python, use the official **SDK**, which builds the request, adds the auth headers, retries on 429 and 5xx, and raises clear exceptions:

```
pip install anthropic
```

Get an API key from **console.anthropic.com** (usage is billed per token, so a few test summaries cost very little) and put it in `.env`:

```
ANTHROPIC_API_KEY=sk-ant-...
```

The code below skips summaries when no key is set, so everything else works without one.

enrich.py: Fetching Pages and Writing Summaries

app/enrich.py:

```
import re

import anthropic
import httpx
from sqlmodel import Session

from app.config import settings
from app.database import engine
from app.models import Bookmark

def fetch_page(url: str) -> tuple[str | None, str]:
    """Download a page and return (title, visible text)."""
    response = httpx.get(url, timeout=10, follow_redirects=True)
    response.raise_for_status()
    html = response.text
    match = re.search(r"<title[^>]*>(.*?)</title>", html, re.IGNORECASE |
    ↪ re.DOTALL)
    title = match.group(1).strip() if match else None
    text = re.sub(r"<(script|style)[^>]*>.*?</\1>", " ", html, flags=re.DOTALL |
    ↪ re.IGNORECASE)
    text = re.sub(r"<[^>]+>", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return title, text
```

```

def summarize(text: str) -> str | None:
    """Ask Claude for a two-sentence summary. Returns None if it can't."""
    if not settings.anthropic_api_key:
        return None
    client = anthropic.Anthropic(api_key=settings.anthropic_api_key, timeout=60)
    response = client.beta.messages.create(
        model="claude-opus-5",
        max_tokens=1024,
        output_config={"effort": "low"},
        betas=["server-side-fallback-2026-07-01"],
        fallbacks="default",
        messages=[{
            "role": "user",
            "content": f"Summarize this web page in two
                ↪ sentences:\n\n{text[:20000]}",
        }],
    )
    if response.stop_reason == "refusal":
        return None
    return "".join(block.text for block in response.content if block.type ==
        ↪ "text")

def enrich_bookmark(bookmark_id: int) -> None:
    """Background job: fill in a bookmark's title and summary."""
    with Session(engine) as session:
        bookmark = session.get(Bookmark, bookmark_id)
        if bookmark is None:
            return
        try:
            title, text = fetch_page(bookmark.url)
        except httpx.HTTPError:
            return # the site is down or slow; keep the bookmark as it is
        if bookmark.title is None:
            bookmark.title = title
        try:
            bookmark.summary = summarize(text)
        except anthropic.APIError:
            pass # summaries are a bonus; never fail the job over one
        session.add(bookmark)
        session.commit()

def get_enricher():
    return enrich_bookmark

```

Walk through the decisions, because these are the habits that make integrations reliable:

- `fetch_page` uses `httpx` with a **timeout** and `follow_redirects=True` (many URLs redi-

rect), and `raise_for_status()` so a 404 page isn't summarized. The regular expressions are a quick way to strip HTML to text; for serious scraping you'd use a parser like BeautifulSoup.

- **summarize** calls the **Messages API**: you send a list of messages (here one user message), a model, and `max_tokens` (a cap on the reply's length), and get back `response.content`, a list of blocks, whose text blocks hold the answer.
 - `model="claude-opus-5"` is a capable current model. Cheaper, faster models exist (such as `claude-haiku-4-5`); for high-volume jobs, compare quality against price and choose. That's a product decision, not a technical one.
 - `output_config={"effort": "low"}` asks the model not to overthink a simple task, which keeps it quick and cheap.
 - `fallbacks="default"` (with its `betas` flag, which is why this uses `client.beta.messages`) means that if the model declines a request, the API automatically retries it on a suitable fallback model. And we still check `stop_reason == "refusal"` in case the whole chain declines.
 - `text[:20000]` limits how much of a very long page we send, which bounds the cost of each call.
- **enrich_bookmark** runs in the background, *after* the request's session has closed, so it **opens its own session**.
- **Failure is contained.** A slow website or an AI error must never break bookmark creation. The bookmark simply stays without a title or summary. Decide for every integration: *what happens when this external API is down?*
- **get_enricher** returns the function. Wrapping it in a dependency looks odd now, but it lets tests swap in a do-nothing version so they don't make real network calls (Chapter 16).

Because `enrich_bookmark` is a plain `def`, FastAPI runs it in a worker thread, so the blocking `httpx.get` and the SDK call don't stall other requests.

Update `create_bookmark` in `app/routers/bookmarks.py`:

```
from fastapi import APIRouter, BackgroundTasks, Depends, HTTPException, Query,
↳ status

from app.enrich import get_enricher

@router.post("", response_model=BookmarkRead, status_code=status.HTTP_201_CREATED)
def create_bookmark(
    data: BookmarkCreate,
    background_tasks: BackgroundTasks,
    user: User = Depends(get_current_user),
    session: Session = Depends(get_session),
    enrich=Depends(get_enricher),
):
```

```

bookmark = Bookmark(url=str(data.url), title=data.title, notes=data.notes,
    ↪ owner_id=user.id)
session.add(bookmark)
session.commit()
session.refresh(bookmark)
background_tasks.add_task(enrich, bookmark.id)
return bookmark

```

Try it: create a bookmark for `https://example.com`. The response has `"title": null`. Wait two seconds and GET it again: the title is `"Example Domain"`, and with a key set, there's a summary.

Async Calls: When They're Worth It

Endpoints that *mainly wait on other APIs* are where `async def` shines. A proxy endpoint that fetches a GitHub profile:

```

import httpx
from fastapi import APIRouter, HTTPException

router = APIRouter(prefix="/github", tags=["github"])

@router.get("/{username}")
async def github_profile(username: str):
    async with httpx.AsyncClient(timeout=10) as client:
        response = await client.get(f"https://api.github.com/users/{username}")
    if response.status_code == 404:
        raise HTTPException(status_code=404, detail="GitHub user not found")
    if response.status_code != 200:
        raise HTTPException(status_code=502, detail="GitHub is unavailable")
    data = response.json()
    return {"name": data["name"], "public_repos": data["public_repos"]}

```

While this request waits on GitHub, the server's event loop handles other requests. Note the **status translation**: GitHub's 404 becomes your 404, and anything else unexpected becomes **502 Bad Gateway** ("an upstream service failed"). Never pass a third-party's raw errors straight through to your clients.

Receiving Webhooks

From Chapter 8: a provider POSTs events to your URL, and you must verify the signature. The usual scheme is an **HMAC**: the provider computes HMAC-SHA256(`shared_secret`, `raw_body`) and sends it in a header; you compute the same and compare.

```

import hashlib
import hmac
import json

from fastapi import APIRouter, BackgroundTasks, Header, HTTPException, Request

router = APIRouter(prefix="/webhooks", tags=["webhooks"])
WEBHOOK_SECRET = b"whsec_from_the_provider_dashboard"    # load from settings in
↳ real code
processed_events: set[str] = set()                        # use a database table in
↳ real code

def handle_payment(data: dict) -> None:
    ...                                                    # e.g. mark the order as
↳ paid

@router.post("/payments")
async def payment_webhook(
    request: Request,
    background_tasks: BackgroundTasks,
    x_signature: str = Header(),
):
    body = await request.body()                            # the raw bytes, exactly
↳ as sent
    expected = hmac.new(WEBHOOK_SECRET, body, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(expected, x_signature):
        raise HTTPException(status_code=401, detail="Invalid signature")

    event = json.loads(body)
    if event["id"] in processed_events:                    # duplicate delivery:
↳ ignore
        return {"received": True}
    processed_events.add(event["id"])

    if event["type"] == "payment.succeeded":
        background_tasks.add_task(handle_payment, event["data"])
    return {"received": True}                               # respond fast

```

- **Sign the raw bytes**, not re-serialized JSON; even a changed space breaks the signature.
- `hmac.compare_digest` compares in constant time, so attackers can't guess the signature byte by byte by measuring response times.
- **Deduplicate by event ID, respond quickly**, and do the slow work in the background.
- Real providers differ slightly (Stripe signs `timestamp.body` and includes the timestamp to block replayed events). Follow each provider's docs, or use their SDK's verification helper.

Integration Habits (the Checklist Clients Pay For)

- **Timeouts** on every outgoing call.
- **Retries** with backoff for 429/5xx/network errors (SDKs often do it; with httpx, do it yourself or use tenacity).
- **Don't make users wait**: slow external work goes in the background.
- **Contain failures**: decide what happens when each dependency is down.
- **Translate errors**: third-party failures become 502/503 from your API, with your own messages.
- **Keys in settings**, never in code; separate test and live keys.
- **Log** each external call's status and duration (Chapter 16), so you can prove whose fault an outage was.

Lab: A Refresh Endpoint

Add a POST `/bookmarks/{bookmark_id}/refresh` endpoint that re-runs enrichment for one of your own bookmarks (use `get_owned_bookmark` and `BackgroundTasks`) and returns **202 Accepted**, the status code meaning “accepted for processing, not done yet”.

Expected results: POST `/bookmarks/1/refresh` returns 202 immediately. A moment later, GET `/bookmarks/1` shows the title (and, with an API key, the summary) filled in. Another user's bookmark ID returns 404.

Summary, Key Terms, and Review Questions

Summary

- `BackgroundTasks` runs work after the response is sent. Use it for slow follow-up work.
- Use official SDKs where they exist (e.g. `anthropic`); use `httpx` otherwise.
- Always set timeouts, contain failures, and translate upstream errors (502) rather than passing them through.
- `async def` + `httpx.AsyncClient` helps endpoints that mainly wait on other APIs.
- Webhooks: verify the HMAC signature over the raw body, deduplicate, respond fast.

Key Terms

Background task, Task queue, SDK, Messages API, Fallback model, HMAC, Constant-time comparison, Deduplication, Webhook. Each term is defined in Appendix D.

Review Questions

1. Why does `enrichment` run as a background task instead of inside the request?
2. Why does `enrich_bookmark` open its own database session?
3. What should your API return when a third-party API it depends on fails, and why?
4. How does HMAC signature verification prove that a webhook is genuine?

Testing and Shipping

An API isn't finished until it's **tested** and **running somewhere others can reach it**.

Automated Tests with pytest and TestClient

Clicking through `/docs` after every change doesn't scale. Automated tests call your API in code and check the results in seconds.

FastAPI's `TestClient` sends requests straight into your app **without running a server**. **pytest** finds and runs test functions.

Two problems to solve first:

1. Tests must not touch your real `bookmarks.db`.
2. Tests must not make real network calls (the web page fetch and the AI API).

Dependency overrides solve both: `app.dependency_overrides[original] = replacement` swaps any dependency during tests. This is the payoff for using dependencies everywhere.

`tests/__init__.py` (empty) and `tests/conftest.py` (pytest loads it automatically; **fixtures** are reusable setup functions that tests request by name):

```
import pytest
from fastapi.testclient import TestClient
from sqlalchemy.pool import StaticPool
from sqlmodel import Session, SQLModel, create_engine

from app.database import get_session
from app.enrich import get_enricher
from app.main import app

@pytest.fixture
def client():
    engine = create_engine(
```

```

        "sqlite://",                                     # an in-memory database
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,                           # share one connection so the
↪ data persists
    )
    SQLAlchemyModel.metadata.create_all(engine)

    def get_test_session():
        with Session(engine) as session:
            yield session

    app.dependency_overrides[get_session] = get_test_session
    app.dependency_overrides[get_enricher] = lambda: (lambda bookmark_id: None) #
↪ no network
    yield TestClient(app)
    app.dependency_overrides.clear()

@pytest.fixture
def auth_headers(client):
    client.post("/auth/register", json={"email": "asha@example.com", "password":
↪ "secret123"})
    response = client.post(
        "/auth/token", data={"username": "asha@example.com", "password":
        ↪ "secret123"}
    )
    token = response.json()["access_token"]
    return {"Authorization": f"Bearer {token}"}

```

Each test gets a fresh, empty, in-memory database. `auth_headers` registers and logs in a user and returns ready-to-use headers.

`tests/test_bookmarks.py`:

```

def test_create_and_read_bookmark(client, auth_headers):
    response = client.post(
        "/bookmarks", json={"url": "https://fastapi.tiangolo.com"},
        ↪ headers=auth_headers
    )
    assert response.status_code == 201
    bookmark = response.json()
    assert bookmark["url"] == "https://fastapi.tiangolo.com/"

    response = client.get(f"/bookmarks/{bookmark['id']}", headers=auth_headers)
    assert response.status_code == 200
    assert response.json()["id"] == bookmark["id"]

def test_invalid_url_is_rejected(client, auth_headers):

```

```

    response = client.post("/bookmarks", json={"url": "not a url"},
↳ headers=auth_headers)
    assert response.status_code == 422

def test_requires_login(client):
    response = client.get("/bookmarks")
    assert response.status_code == 401

def test_cannot_see_other_users_bookmarks(client, auth_headers):
    created = client.post(
        "/bookmarks", json={"url": "https://example.com"}, headers=auth_headers
    ).json()

    client.post("/auth/register", json={"email": "ravi@example.com", "password":
↳ "secret456"})
    token = client.post(
        "/auth/token", data={"username": "ravi@example.com", "password":
↳ "secret456"}
    ).json()["access_token"]

    response = client.get(
        f"/bookmarks/{created['id']}", headers={"Authorization": f"Bearer
↳ {token}"})
    assert response.status_code == 404

def test_update_and_delete(client, auth_headers):
    created = client.post(
        "/bookmarks", json={"url": "https://example.com"}, headers=auth_headers
    ).json()

    response = client.patch(
        f"/bookmarks/{created['id']}", json={"notes": "read later"},
↳ headers=auth_headers
    )
    assert response.json()["notes"] == "read later"

    response = client.delete(f"/bookmarks/{created['id']}", headers=auth_headers)
    assert response.status_code == 204
    response = client.get(f"/bookmarks/{created['id']}", headers=auth_headers)
    assert response.status_code == 404

```

Run from the project root:

```
pytest
```

```
.....
```

```
[100%]
```

```
5 passed in 0.32s
```

What to test first, in priority order:

1. **The happy path** of each endpoint (create, read, update, delete).
2. **Security**: unauthenticated → 401; **another user’s data** → **404**. These are the tests that prevent disasters.
3. **Validation**: bad input → 422.
4. **Business rules**: duplicate email → 409, and similar.



Run tests before every commit. When you find a bug, first write a test that reproduces it, then fix it. It will never come back unnoticed.

Logging: Seeing What Happened

In production you can’t watch the terminal. **Logs** are how you find out what happened. Uvicorn already logs every request (method, path, status). Add your own for important events:

```
import logging

logger = logging.getLogger(__name__)

logger.info("Enriched bookmark %s in %.1fs", bookmark_id, elapsed)
logger.warning("Page fetch failed for %s: %s", url, error)
```

Put `logging.basicConfig(level=logging.INFO)` near the top of `app/main.py` so these messages appear. Log external-call failures and important business events. **Never log passwords, tokens, or API keys.**

Docker: Packaging Your App

A **container** bundles your app with its exact Python version and packages, so it runs identically on your laptop and on any server. **Docker** builds and runs containers. The recipe is a `Dockerfile` in the project root:

```
FROM python:3.12-slim

WORKDIR /code
```

```

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

RUN useradd --create-home appuser && chown -R appuser /code
USER appuser

EXPOSE 8000
CMD ["fastapi", "run", "app/main.py", "--port", "8000"]

```

Line by line: start from an official slim Python image, install requirements (copied first so Docker can **cache** this slow step when only your code changes), copy the code, create a non-root user that owns the app folder (safer than running as root, and it can still write files such as a SQLite database there), and start the server with **fastapi run**, the production mode (no auto-reload, listens on all network interfaces so it's reachable from outside the container).

Add a `.dockerignore` so secrets and junk don't get copied into the image:

```

.venv
.env
*.db
__pycache__
tests

```

Build and run (with Docker Desktop installed):

```

docker build -t bookmarks-api .
docker run -p 8000:8000 --env-file .env bookmarks-api

```

`-p 8000:8000` connects port 8000 on your machine to port 8000 in the container (Chapter 2's ports again). `--env-file .env` passes your settings in **at run time**; they're never baked into the image.

Deploying

Beginner-friendly hosting platforms (**Render**, **Railway**, **Fly.io**, **Google Cloud Run**, and others) all work roughly the same way:

1. **Push your code to GitHub** (with `.env` git-ignored!).
2. **Create a web service** on the platform and connect the repository. It detects the Dockerfile (or you give it a start command such as `fastapi run app/main.py --port $PORT`; many platforms tell your app which port to use through a `PORT` variable).

3. **Set environment variables** in the platform's dashboard: `JWT_SECRET` (a new long random value, different from your local one), `ANTHROPIC_API_KEY`, `CORS_ORIGINS` (e.g. `["https://myfrontend.com"]`), `DATABASE_URL`.
4. **Add a managed PostgreSQL database** from the same platform, and put its connection URL in `DATABASE_URL`. For `SQLModel`, the URL looks like `postgresql+psycopg://user:password@host:5432/dbname`, and you add `psycopg[binary]` to `requirements.txt`. **Don't use SQLite in production** on these platforms: their disks are usually wiped on every redeploy, taking your data with them.
5. **Set the health check path** to `/health`.
6. **Deploy.** You get a public HTTPS URL like `https://bookmarks-api.onrender.com`. HTTPS is handled by the platform.

Then: add a `Production` environment in Postman with that `base_url`, and run your collection against it. Your docs are live at `https://.../docs`, which is great to send to a client.

Every push to GitHub can trigger a redeploy automatically. When you're ready, add a **CI** pipeline (e.g. GitHub Actions) that runs `pytest` on every push and blocks broken code from deploying.

Before You Call It Production-Ready

- ☐ Tests pass, including auth and ownership tests
- ☐ All secrets in environment variables; `.env` not in Git
- ☐ PostgreSQL, not SQLite; backups enabled on the database
- ☐ Specific CORS origins
- ☐ `/health` endpoint configured
- ☐ Logs visible in the platform's dashboard
- ☐ Rate limiting on login (platform, gateway, or `slowapi`)
- ☐ Alembic migrations set up before the schema changes with real data in it

Lab: More Tests, Then Deploy

Write a test for `POST /auth/register` with a duplicate email (expect 409), and one for a too-short password (expect 422). Then deploy the API to any platform above, and send the `/docs` link to a friend.

Expected results: `pytest` reports 7 passed. After deploying, `https://<your-app>/health` returns `{"status": "ok"}` and `/docs` loads in the browser.

Summary, Key Terms, and Review Questions

Summary

- `TestClient` + `pytest` test your API without a server; `dependency_overrides` swaps in a test database and fake integrations.
- Test security (401s, other users' data) as seriously as the happy path.
- Log important events and failures, never secrets.
- A `Dockerfile` packages the app; settings are passed at run time.
- Deploy: push to GitHub → platform builds → set environment variables → managed Postgres → HTTPS URL.

Key Terms

pytest, **TestClient**, **Fixture (pytest)**, **Dependency override**, **Docker / container**, **Dockerfile**, **Image (Docker)**, **Health check**, **Managed database**, **CI (Continuous Integration)**. Each term is defined in Appendix D.

Review Questions

1. How do dependency overrides let tests avoid the real database and the network?
2. Which tests matter most for security?
3. Why pass settings at `docker run` time instead of building them into the image?
4. Why shouldn't you use SQLite on most hosting platforms?

PART IV

From Learner to Earner

What Clients Pay For, and What to Learn Next

You can now use APIs, debug them, test them in Postman, and build, secure, test, and deploy one with FastAPI. That's a real, sellable skill set. This chapter is about turning it into work.

The Kinds of API Work People Pay For

These come up constantly, in freelance marketplaces, agency work, and junior backend roles:

1. **Backends for apps (MVPs).** A founder has a mobile or web app design and needs the API behind it: users, logins, a few resources, CRUD, deployment. This is exactly Part 3, with different nouns.
2. **Integrations.** “When someone pays on Stripe, create the customer in our CRM, add them to our email list, and post in Slack.” You receive webhooks, call three APIs, handle failures. Chapters 7, 8, and 15.
3. **AI features.** “Summarize these documents”, “classify incoming support tickets”, “let users chat with our knowledge base”. An API that accepts input, calls an LLM API, and stores or returns the results. It's one of the fastest-growing kinds of work, and FastAPI is the default framework for it.
4. **Automation and data sync.** Scripts or small services that pull data from one API on a schedule (with pagination and rate limits) and push it into a spreadsheet, database, or dashboard.
5. **Fixing and extending existing APIs.** “Our API is slow / broken / needs a new endpoint.” You'll read someone else's FastAPI or Flask code, use the Network tab and Postman to reproduce the issue, and add tests. Often the easiest work to win at first, because the client already has something.

Notice what they share: very few are about clever algorithms. They're about **reliability**: validation, auth, error handling, timeouts, tests, clear docs, and a deployed URL that works.

That's what you practised.

Your Portfolio: Three Projects

Clients and employers want evidence. Build three projects that match the work above:

1. **The Bookmarks API** from this book, deployed, with a live `/docs` link.
2. **An integration project.** For example: a small shop backend using **Stripe in test mode**: create a checkout, receive the `checkout.session.completed` webhook, verify the signature, and record the order. Or: a form-submission API that saves leads and posts them to Slack.
3. **An AI project.** For example: a support-ticket API that accepts a ticket, asks an LLM to classify its category and urgency as structured JSON, stores it, and exposes filtered lists. Or: a document API: upload a PDF, extract its text, and answer questions about it.

For each project, have:

- A **GitHub repository** with a clear **README**: what it does, the endpoints, how to run it locally (`.env.example`, `pip install -r requirements.txt`, `fastapi dev`), and how to run the tests.
- A **live deployment** and its `/docs` URL.
- A **Postman collection** (exported to the repo) with an environment, a login request that saves the token, and tests.
- **Tests** that include auth and ownership checks.
- Optionally, a 2-minute screen recording walking through it in `/docs` or Postman.

A client can judge that in five minutes. It says more than any certificate.

How to Approach a Client Project

- **Clarify the resources and actions first.** Before code, write the endpoint list (METHOD /path → purpose → status codes), as you did in Chapter 4's exercise. Agree on it with the client. It becomes both the plan and the scope.
- **Ask about the integrations early.** Which external APIs? Do test/sandbox accounts exist? Who owns the API keys? Integration unknowns are where timelines slip.
- **Deliver in small milestones:** e.g. (1) data model + CRUD deployed, (2) auth, (3) integrations, (4) tests + handover.
- **Hand over properly:** the repo, the live URL, the `/docs` link, the Postman collection, a README, and a list of every environment variable. Professional handover is memorable, and it brings repeat work.

What to Learn Next (in This Order)

1. **Git and GitHub, properly:** branches, pull requests, meaningful commits. Every job assumes it.
2. **SQL and PostgreSQL:** SELECT, JOIN, GROUP BY, indexes. The ORM writes SQL, but you'll debug it.
3. **Alembic migrations:** the moment you have real data.
4. **Relationships in SQLAlchemy:** one-to-many and many-to-many (e.g. bookmarks with tags).
5. **A real payment integration** (Stripe) and **a real OAuth login** (Google via a library or an auth service).
6. **Deeper LLM API work:** streaming responses to clients (Server-Sent Events), structured JSON output, and **tool use** (letting the model call your functions).
7. **CI with GitHub Actions:** run tests on every push.
8. **Background job queues and Redis**, when a project needs long-running jobs or caching.
9. **Async database access**, when you have a measured performance need (not before).

What to Deliberately Skip for Now

Microservices, Kubernetes, GraphQL servers, gRPC, event streaming platforms, learning Django and Flask as well, and rewriting working projects in the newest framework. Each has its place; none of them will get you your first ten projects. Depth in one stack (FastAPI + Postgres + Docker + one cloud platform) beats a shallow tour of ten.

A Six-Week Plan

Week	Focus	Output
1	Re-read Parts 1–2; do every “Try it”; build Postman collections for GitHub and one other public API	Two collections, comfortable with curl and httpx
2	Build the Bookmarks API through Chapter 13, typing everything yourself	Working CRUD with a database
3	Chapters 14–16: auth, integrations, tests, Docker, deploy	Live URL, passing tests
4	Portfolio project 2 (integration, Stripe test mode)	Deployed, with README and collection
5	Portfolio project 3 (AI feature)	Deployed, with README and collection

Week	Focus	Output
6	Polish READMEs, record walkthroughs, start applying or bidding	Profile with three live projects

Exercises

1. Write the endpoint table for portfolio project 2: each METHOD /path, its purpose, its success code, and its likely error codes.
2. Draft the README outline for your Bookmarks API using the checklist in the portfolio section.
3. Put your week 1 tasks from the six-week plan into your calendar.

Summary, Key Terms, and Review Questions

Summary

- Paid API work is mostly app backends, integrations, AI features, automation, and fixing existing APIs.
- Build three deployed portfolio projects with READMEs, tests, /docs links, and Postman collections.
- Agree on the endpoint list before coding; deliver in milestones; hand over thoroughly.
- Go deep on one stack before going wide.

Key Terms

MVP (Minimum Viable Product), Integration, Portfolio, README, Milestone, Handover. Each term is defined in Appendix D.

Review Questions

1. Name the five common kinds of paid API work.
2. What should every portfolio project include?
3. Why agree on the endpoint list with a client before writing code?
4. Which topics does this book suggest you skip for now, and why?

Appendices

Quick references to keep open while you work: status codes, headers, FastAPI syntax, and a glossary of every term in the book.

Status Codes Cheat Sheet

The Status Codes You'll Meet

Code	Name	Meaning / when to use
200	OK	Success with a body (GET, PATCH, PUT)
201	Created	A POST created a resource; optionally include Location
202	Accepted	Accepted for processing, not finished (background jobs)
204	No Content	Success, no body (DELETE)
301	Moved Permanently	Resource permanently at a new URL
302 / 307	Found / Temporary Redirect	Temporarily elsewhere
304	Not Modified	Cached copy is still valid
400	Bad Request	Malformed request, or invalid in a general way
401	Unauthorized	Not authenticated: missing/invalid/expired credentials
403	Forbidden	Authenticated, but not allowed
404	Not Found	Doesn't exist, or you mustn't know it exists
405	Method Not Allowed	Path exists but not with this method (e.g. DELETE on /health)
409	Conflict	Clashes with current state (duplicate, version conflict)
413	Content Too Large	Body/upload too big
415	Unsupported Media Type	Wrong Content-Type (e.g. sent form data where JSON expected)
422	Unprocessable Content	Valid syntax, invalid data. FastAPI's validation errors
429	Too Many Requests	Rate limited; check Retry-After

Code	Name	Meaning / when to use
500	Internal Server Error	Unhandled exception in the server's code
502	Bad Gateway	A server/proxy got a bad response from the service behind it
503	Service Unavailable	Down or overloaded; retry later
504	Gateway Timeout	A proxy timed out waiting for the service behind it

The Debugging Rule

4xx → check your request. 5xx → check the server (or retry later).

Headers Cheat Sheet

Request Headers

Header	Example	Purpose
Authorization	Bearer eyJ... / Basic YXNo...	Credentials
Content-Type	application/json	Format of the body you're sending
Accept	application/json	Format you want back
User-Agent	my-app/1.0	Identifies the client program
X-API-Key	abc123	API key (custom header, common)
Idempotency-Key	5f1c2a9e-...	Makes retrying a POST safe
If-None-Match	"abc"	"Only send it if it changed since this version" (caching)
Origin	http://localhost:3000	Sent by browsers; used for CORS
Cookie	session=...	Browser session cookie

Response Headers

Header	Example	Purpose
Content-Type	application/json	Format of the body
Location	/bookmarks/17	URL of the created resource (201) or redirect target
Retry-After	30	Seconds to wait (429, 503)
WWW-Authenticate	Bearer	How to authenticate (with 401)
X-RateLimit-Limit / -Remaining / -Reset	5000 / 12 / 1790361130	Rate-limit status
X-Request-ID / request-id	req_8a7f...	Unique request ID; quote it in bug reports
Link	<...?page=2>; rel="next"	Pagination links
ETag	"abc"	Version identifier of the response (caching)
Cache-Control	max-age=60	How long the response may be cached
Access-Control-Allow-Origin	https://myapp.com	CORS: which origins may read this
Set-Cookie	session=...; HttpOnly; Secure	Sets a browser cookie

The X- Prefix

An X- prefix marks a custom, non-standard header. Check the API's own docs for its meaning.

FastAPI Cheat Sheet

Endpoints, Parameters, and Responses

```

from fastapi import (FastAPI, APIRouter, Depends, HTTPException, Query,
                    Header, Form, File, UploadFile, BackgroundTasks, status)
from fastapi.middleware.cors import CORSMiddleware
from fastapi.testclient import TestClient

app = FastAPI(title="My API", version="1.0.0")
router = APIRouter(prefix="/items", tags=["items"])

# Methods: one decorator per endpoint function
# @router.get("") list
# @router.post("", status_code=201) create
# @router.get("/{item_id}") read one
# @router.patch("/{item_id}") partial update
# @router.delete("/{item_id}", status_code=204) delete

# Where arguments come from (arguments without defaults first, as usual in Python)
@router.patch("/{item_id}", response_model=ItemRead) # response_model filters +
↳ documents output
def update_item(
    item_id: int, # path, because {item_id} is
    ↳ in the path
    data: ItemUpdate, # JSON body, because it's a
    ↳ Pydantic model
    background_tasks: BackgroundTasks, # run work after responding
    q: str | None = None, # optional query parameter
    limit: int = Query(default=20, ge=1, le=100), # query parameter with rules
    user_agent: str | None = Header(default=None), # the User-Agent header
    session: Session = Depends(get_session), # a dependency
):
    changes = data.model_dump(exclude_unset=True) # PATCH: only the fields the
    ↳ client sent
    ...

```

```

    raise HTTPException(status_code=404, detail="Not found")    # error response

# Forms and uploads (instead of a JSON body, not together with one)
@router.post("/upload")
async def upload(name: str = Form(), file: UploadFile = File()):
    content = await file.read()

# Protect every endpoint in a router
router = APIRouter(prefix="/admin", dependencies=[Depends(get_current_user)])

# Wire up
app.include_router(router)
app.add_middleware(CORSMiddleware, allow_origins=["https://myapp.com"],
                  allow_methods=["*"], allow_headers=["*"])

# Tests
client = TestClient(app)
app.dependency_overrides[get_session] = get_test_session

```

Commands and URLs

Command	Purpose
<code>fastapi dev app/main.py</code>	Run in development (auto-reload, localhost:8000)
<code>fastapi run app/main.py</code>	Run in production mode
<code>pytest</code>	Run tests
<code>/docs, /redoc, /openapi.json</code>	Interactive docs, read-only docs, raw OpenAPI spec

def vs. async def

Use `async def` only if you await async libraries inside; otherwise `def` (FastAPI runs it in a thread). Never put blocking calls in `async def`.

Glossary

Glossary of Terms

Access token: A short-lived credential sent with requests (usually as a bearer token) to prove you're authorized.

Alembic: A tool for database migrations with SQLAlchemy/SQLModel.

API: An agreed way for one program to request data or actions from another.

API call: One request sent to an API and the response that comes back.

API gateway: Infrastructure in front of APIs that handles routing, auth checks, and rate limiting.

API key: A secret string identifying the calling program; sent with each request.

APIRouter: FastAPI's way to group related endpoints in their own file.

Argon2: A modern password-hashing algorithm, deliberately slow so that guessing passwords is expensive.

ASGI: The standard interface between Python async web servers (Uvicorn) and apps (FastAPI).

async / await: Python syntax for code that can pause while waiting on I/O, letting other work run.

Authentication: Proving who you are. Failure → 401.

Authorization: Deciding what you're allowed to do. Failure → 403. Also the name of the HTTP header carrying credentials.

Authorization Code flow: The OAuth 2.0 flow where a user approves access and the app exchanges a one-time code for tokens.

Background task: Work FastAPI runs after sending the response.

Backoff (exponential): Waiting progressively longer between retries (1s, 2s, 4s...).

Base URL: The common start of every endpoint URL for an API, e.g. `https://api.github.com`.

Base64: An encoding (not encryption) that represents any data using plain characters.

Basic auth: Sending `username:password` Base64-encoded in the Authorization header.

Bearer token: A token sent as `Authorization: Bearer <token>`; whoever holds it gets access.

Body: The data part of a request or response, usually JSON.

BOLA (Broken Object Level Authorization): An API flaw where a logged-in user can reach other users' objects just by changing an ID. The most common serious API vulnerability.

Certificate (TLS): A server's digital ID proving its domain, used by HTTPS.

CI (Continuous Integration): Automatically running tests and other checks on every push.

Claim: A statement inside a JWT's payload, such as `sub` (who the token is for) or `exp` (when it expires).

Client: The program that sends a request.

Client Credentials flow: The OAuth 2.0 flow for machine-to-machine access, with no user involved.

Client ID: In OAuth, the public identifier of an application.

Client secret: In OAuth, the application's private password. Server-side only.

Collection (Postman): A saved, organised group of requests.

Collection Runner (Postman): Runs every request in a collection in order and reports the test results.

Commit (database): Saving the changes made in a database session permanently.

Constant-time comparison: Comparing secrets so that the time taken reveals nothing about how much matched (`hmac.compare_digest`).

Content-Type: The header saying what format a body is in.

Cookie: A small value a browser stores and sends back automatically with every request to the same site.

Coroutine: A function defined with `async def`; it can pause at `await` points.

CORS: A browser rule restricting web pages from reading responses from other origins unless allowed by the server.

CRUD: Create, Read, Update, Delete, the basic data operations.

curl: A command-line tool for sending HTTP requests.

Cursor pagination: Paging with an opaque pointer to where the previous page ended.

Decorator: Python syntax (`@something`) that wraps or registers a function; FastAPI's `@app.get(...)` registers an endpoint.

Deduplication: Recognising and ignoring repeated deliveries of the same event, usually by its ID.

Dependency (FastAPI): A function FastAPI runs before your endpoint to provide something it needs, via `Depends`.

Dependency injection: Declaring what a function needs and letting the framework provide it.

Dependency override: Replacing a FastAPI dependency with a different function, usually in tests.

DevTools: The browser's built-in developer tools; the Network tab shows every API call a page makes.

DNS: The system that translates domain names into IP addresses.

Docker / container: A way to package an app with its runtime and dependencies so it runs the same anywhere.

Dockerfile: The recipe Docker follows to build an image of your app.

Documentation (API docs): The description of what an API offers and how to call it: the “menu”.

Endpoint: One operation of an API: a method plus a path, e.g. `GET /bookmarks`.

Engine (database): The object that manages connections to a database; created once per application.

Environment (Postman): A named set of variable values, e.g. Local vs Production.

Environment variable: A setting provided to a program by its operating environment, used for configuration and secrets.

Event loop: The engine that runs async code, switching between tasks at `await` points.

Fallback model: A model an AI API automatically retries a request on if the first model declines it.

FastAPI: A Python framework for building web APIs from ordinary type-hinted functions.

Fixture (pytest): A reusable setup function that tests request by naming it as a parameter.

Foreign key: A database column referencing a row in another table.

Form data: A body of `key=value` pairs (`application/x-www-form-urlencoded`), as sent by HTML forms.

GraphQL: An API style where clients send queries specifying exactly which fields they want.

gRPC: A fast binary API protocol, used mainly between internal services.

Handover: Delivering a finished project to a client: code, live URL, docs, collection, and a list of settings.

Hash (password): A one-way transformation of a password; stored instead of the password itself.

Header: A `Name: value` line carrying metadata about a request or response.

Health check: A simple endpoint (like `/health`) that tools call to confirm an app is running.

HMAC: A signature computed from a message and a shared secret; used to verify webhooks.

Host: The domain or IP address part of a URL.

HTTP: The protocol defining the format of web requests and responses.

HTTPS: HTTP encrypted with TLS.

httpx: A modern Python library for making HTTP requests (sync and async).

Idempotency key: A unique ID sent with a request so retries don't repeat its effect.

Idempotent: Having the same effect whether performed once or many times (GET, PUT, DELETE).

Image (Docker): A packaged, runnable snapshot of an app and its environment. A running image is a container.

Integration: Code that connects two or more systems through their APIs.

IP address: A numeric network address of a computer, e.g. `140.82.112.6`.

Jitter: Randomness added to retry delays so that many clients don't retry at the same instant.

JSON: The text data format used by most APIs, mapping to Python dicts and lists.

JWT: JSON Web Token: a signed token with a readable payload of claims, such as user ID and expiry.

Lifespan: FastAPI's startup/shutdown hook.

Load balancer: Infrastructure spreading incoming requests across multiple servers.

localhost: The name for "this same computer" (`127.0.0.1`).

Managed database: A database run for you by a hosting provider, including backups and updates.

Messages API: Anthropic's API for sending messages to Claude models and receiving their replies.

Method: The HTTP verb: GET, POST, PUT, PATCH, DELETE.

Middleware: Code wrapping every request and response, e.g. for CORS or logging.

Migration: A versioned script that changes a database's structure.

Milestone: A checkpoint in a project with an agreed deliverable.

Multipart: A body format (`multipart/form-data`) that can carry files; used for uploads.

MVP (Minimum Viable Product): The smallest version of a product that is useful enough to launch.

OAuth 2.0: A standard for granting an app limited access to a user’s data without sharing their password.

Offset/limit pagination: Paging by skipping N records and returning M.

OpenAPI: A standard machine-readable description of an API; FastAPI generates it at `/openapi.json`.

OpenID Connect (OIDC): A login layer on top of OAuth 2.0 (“Log in with Google”).

Origin: Scheme + host + port, e.g. `http://localhost:3000`; the unit browsers use for CORS.

ORM: Object-Relational Mapper: work with database rows as Python objects.

Pagination: Returning a large result set one page at a time.

Path: The part of a URL after the host, identifying a resource, e.g. `/bookmarks/17`.

Path parameter: A variable part of the path, e.g. 17 in `/bookmarks/{bookmark_id}`.

PKCE: An OAuth add-on protecting apps that can’t keep a client secret.

Polling: Repeatedly asking an API whether something has changed.

Port: A number identifying which program on a computer receives a connection (443, 8000...).

Portfolio: A set of finished, public projects that shows what you can build.

Postman: A desktop app for building, saving, and testing API requests.

Preflight request: An `OPTIONS` request a browser sends first, to ask a server whether a cross-origin request is allowed.

Primary key: A column uniquely identifying each row in a table (usually `id`).

Problem Details: A standard JSON format for API errors, with `type`, `title`, `status`, and `detail` fields.

Pydantic: A Python library for declaring data shapes and validating data.

pytest: The standard Python testing tool.

Query parameter: A `key=value` option after `?` in a URL.

Query string: The part of a URL after `?`: `key=value` pairs joined by `&`.

Rate limit: A cap on how many requests a client may make in a period.

README: The front-page document of a repository: what the project is and how to run it.

Redirect URI: In OAuth, the pre-registered URL the provider sends the user back to.

Refresh token: A long-lived OAuth token used to get new access tokens.

Relational database: A database that stores data in tables of rows and columns and is queried with SQL.

Request: The message a client sends: method, URL, headers, and body.

Request ID: A unique identifier for one request, returned in a header so it can be traced through logs.

Requirements file: `requirements.txt`: the exact packages (and versions) a project needs.

Resource: A “thing” an API exposes, like users or bookmarks.

Response: The message a server returns: status code, headers, and body.

response_model: FastAPI setting that filters, validates, and documents what an endpoint returns.

REST: An API style built around resources in URLs and HTTP methods as actions.

Reverse proxy: A server in front of your app that forwards requests to it (and often handles HTTPS).

Routing: Matching a request’s method and path to the code that handles it.

Safe method: A method that changes nothing on the server, such as GET.

Salt: Random data mixed into a password hash so identical passwords hash differently.

Scheme: The first part of a URL (`https`, `http`), saying how to communicate.

Scope: In OAuth, a named permission such as `calendar.readonly`.

Script (Postman): JavaScript that runs before a request is sent (pre-request) or after the response arrives (post-response); used for tests and chaining.

SDK: A library that wraps an API’s HTTP calls in convenient functions for a language.

Server: The program that receives requests and sends responses.

Session (database): A unit of work with the database: add, query, commit.

Session (web): Server-side login state, identified by a cookie.

Settings (pydantic-settings): Configuration loaded from environment variables and `.env` into a validated Pydantic model.

SOAP: An older, XML-based API style with strict contracts, still found in banking, insurance, and government systems.

SQLite / PostgreSQL: A file-based database for development / a full database server for production.

SQLModel / SQLAlchemy: Python ORMs; SQLModel is built on SQLAlchemy and Pydantic.

SSE (Server-Sent Events): A way for a server to stream data to a client over one open response.

Starlette: The web toolkit FastAPI is built on.

Stateless: Each request carries everything needed; the server keeps no per-client conversation state.

Status code: The three-digit result of a response (200, 404, 500...).

Swagger UI: The interactive documentation FastAPI serves at `/docs`.

Table (database): A set of rows that share the same columns; usually one table per kind of record.

Task queue: A system that runs background jobs outside the web process, reliably (Celery, RQ, ARQ).

TestClient: FastAPI's tool for calling your app in tests without running a server.

Timeout: A limit on how long to wait for a response.

TLS: The encryption protocol behind HTTPS.

Tunnel: A tool (such as ngrok) that gives your local server a temporary public URL.

Type hint: A label for a value's type (`x: int`); FastAPI uses them for validation.

URL: A full web address: scheme, host, port, path, query.

Uvicorn: The ASGI server that runs FastAPI apps.

Validation: Checking data against rules (types, lengths, formats) before using it.

Variable (Postman): A named value written as `{{name}}` in requests and substituted when the request is sent.

Versioning: Keeping old API behaviour available (e.g. `/v1/`) while introducing changes.

Virtual environment: A per-project folder of installed Python packages.

Web API: An API that programs call over a network, usually using HTTP.

Webhook: An HTTP request a provider sends to your URL when an event happens.

WebSocket: A persistent two-way connection for real-time communication.

Workspace (Postman): The space in Postman that holds your collections and environments.

X- header: A custom, non-standard HTTP header, e.g. `X-API-Key`.

Index

A

- access token, 42
- Alembic, 77
- API, 2
- API call, 4
- API gateway, 9
- API key, 36, 39
- APIRouter, 80
- Argon2, 86
- ASGI, 59
- async / await, 54, 57
- authentication, 30, 39, 44
- authorization, 36, 38, 39, 44

B

- base URL, 30
- base64, 40
- basic auth, 36
- bearer token, 36, 92
- body, 10, 14, 15, 34

C

- certificate (TLS), 8
- CI (Continuous Integration), 108, 113
- claim, 41
- client, 2
- client ID, 41, 42
- client secret, 41, 42
- collection (Postman), x, 32, 35, 38, 92
- collection Runner (Postman), 37
- commit (database), 72

- cookie, 41
- CORS, 92
- CRUD, 22
- curl, x, 27

D

- decorator, 60
- dependency (FastAPI), 59
- dependency injection, 74
- dependency override, 103
- DevTools, 26
- DNS, 7
- Docker / container, 106
- documentation (API docs), 2

E

- endpoint, 30
- engine (database), 72
- environment (Postman), 35, 38
- event loop, 57

F

- fixture (pytest), 103
- foreign key, 71, 87
- form data, 20, 89

G

- GraphQL, 23
- gRPC, 23

H

- hash (password), 86

header, 10, 14, 15, 26, 29, 34, 41
HMAC, 99
httpx, 28

I

idempotency key, 48
idempotent, 15
integration, 111

J

jitter, 48
JSON, 10, 19, 34
JWT, 41

L

lifespan, 76
load balancer, 9

M

messages API, 98
method, 14
middleware, 93
migration, 77
multipart, 20

O

OAuth 2.0, 36
OpenAPI, 30, 37, 62
OpenID Connect (OIDC), 43
origin, 92
ORM, 71

P

pagination, 30
path parameter, 61
PKCE, 42
port, 7
primary key, 71
problem Details, 49
Pydantic, 54, 55, 59
pytest, 103

Q

query parameter, 28

R

rate limit, 30
README, 112
redirect URI, 42
refresh token, 42
relational database, 71
request, 2, 33
request ID, 49
resource, 22
response, 2, 33
REST, 22
reverse proxy, 9
routing, 9, 60

S

salt, 86
scope, 35, 42
script (Postman), 10, 36
SDK, 30, 96
server, 2, 59, 93
session (database), 41, 72
settings (pydantic-settings), 79
SOAP, 23
SQLite / PostgreSQL, 71
SQLModel / SQLAlchemy, 71
Starlette, 59
stateless, 23
status code, 15
Swagger UI, 62

T

table (database), 71
task queue, 95
TestClient, 103
timeout, 97, 101
type hint, 54

U

URL, 6

Uvicorn, 59

V

validation, 106

variable (Postman), 35

virtual environment, 60

W

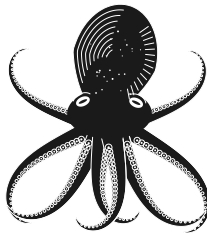
web API, 4

webhook, 23, 49

WebSocket, 23

workspace (Postman), 33

Colophon



The animal on the cover of *APIs to FastAPI* is the common octopus (*Octopus vulgaris*), which lives on rocky sea floors in warm and temperate coastal waters, and is especially familiar from the Mediterranean and the eastern Atlantic.

It is a fitting animal for a book about connecting systems. An octopus has eight arms that work largely on their own: about two-thirds of its roughly 500 million neurons are in its arms rather than its brain, and each arm can touch, taste, and grip through its suckers while the others do something else entirely. Yet the arms serve one body, coordinated through a simple, dependable interface. That is not a bad description of a well-designed API.

The common octopus has three hearts and blue, copper-based blood. It can change the colour and even the texture of its skin in a fraction of a second, using pigment cells called chromatophores and small muscular bumps called papillae. It is a famously good problem-solver and escape artist, and lives for only a year or two.

The cover illustration is an original woodcut-style drawing made for this book, in the tradition of the engraved animals on classic technical-book covers. The cover fonts are EB Garamond; the text font is Crimson Pro; the heading font is Source Sans 3; and the code font is Source Code Pro.

APIs to FastAPI

APIs are how software talks to software, and knowing how to use and build them is one of the most practical skills a developer can have. This book starts from the very beginning (what an API actually is, and what really happens when you make an API call) and takes you all the way to building, securing, testing, and deploying your own API with FastAPI.

It is deliberately short and practical. Instead of every corner of every standard, you get the concepts that matter, in the order you need them, and one real project built step by step.

You'll learn how to:

- Understand HTTP, URLs, ports, headers, status codes, and JSON well enough to debug anything
- Make API calls with the browser, curl, Python, and Postman
- Work with API keys, tokens, JWTs, and OAuth 2.0 (client IDs, secrets, and scopes)
- Handle pagination, rate limits, retries, errors, and webhooks like a professional
- Build a real FastAPI service with a database, logins, background jobs, and AI summaries
- Test it, package it with Docker, and deploy it

Every chapter ends with a hands-on lab, a summary, and review questions, and all of the code has been run and tested.